

Multi-homogeneous XL

Kai-Chun Ning¹, Lars Ran², and Simona Samardjiska²

¹ Max Planck Institute for Security and Privacy, Bochum, Germany
`kai-chun.ning@mpi-sp.org`

² Radboud University, Nijmegen, Netherlands
`{lran,simonas}@cs.ru.nl`

Abstract. Algebraic cryptanalysis is an important and versatile tool in the evaluation of the security of various cryptosystems especially in multivariate cryptography. Its effectiveness can be determined by analyzing the Polynomial System Solving problem (PoSSo). However, the polynomial systems arising from cryptanalytic algebraic models often exhibit structure that is crucial for the solving complexity and is often not well understood.

In this paper we turn our focus to multi-homogeneous systems that very often arise in algebraic models. Despite their overwhelming presence, both the theory and the practical solving methods are not complete. Our work fills this gap.

We develop a theory for multi-homogeneous systems that extends the one for regular and semi-regular sequences. We define “border-regular” systems and provide exact statements about the rank of a specific submatrix of the Macaulay that we associate to these systems. We then use our theoretical results to define Multi-homogeneous XL - an algorithm that extends XL to the multi-homogeneous case. We further provide fully optimized implementation of Multi-homogeneous XL that uses sparse linear algebra and can handle a vast parameter range of multi-homogeneous systems. To the best of our knowledge this is the first implementation of its kind, and we make it publicly available.

Keywords: Parallel implementation · Polynomial system solving · Algebraic cryptanalysis

1 Introduction

Solving systems of non-linear polynomial equations, typically referred to as PoSSo (Polynomial System Solving) problem or MQ (Multivariate Quadratic) problem when the degree of the equations is two, is a fundamental problem in computational algebra. While easy-to-formulate, it is a hard problem in general (actually NP-hard [38]), and for generic instances, the best algorithms are exponential. At the extremes – when the systems are extremely overdetermined or

This research has been partially supported by the Dutch government through the NWO grant OCNW.M.21.193 (ALPaQCa).

underdetermined – PoSSo – can be solved in polynomial time [40,56]. For example, in the very overdetermined case, it is possible to *linearize* the system, and then basic linear algebra can be used to find a solution.

The generic overdetermined case can be treated with a similar idea in mind – First find enough elements in the ideal generated by the system at a certain degree by multiplying all polynomials by monomials; then linearize the new system by treating each monomial as an independent variable. Thus, the problem of finding the solution as an element in the ideal generated by the system of polynomials is turned into a simpler problem of performing basic linear algebra. This approach was first proposed by Buchberger [19], refined by Lazard in [45], and later extended into Gröbner bases algorithms [28,29,30,32,13,10]. In parallel, it was reinvented by Courtois, Klimov, Patarin, and Shamir in [24] where they introduced the acronym XL for extended linearization and later extended to variants in [61,48,39].³

The XL algorithm, or more generally, Gröbner bases algorithms work quite predictably for *semi-regular* polynomial systems which generate ideals with only trivial relations (syzygies) among their elements. We can precisely predict the Hilbert series of these systems, and use them to make good estimates of the runtime of the algorithms. Turns out, semi-regularity is a plausible assumption for polynomial systems generated uniformly at random.

In cryptography, however, things are often more complicated. Multivariate cryptosystems rarely involve generic systems in their designs (apart from a handful of schemes that provably rely on the MQ problem [12,42,21,22,14,11]), but rather systems that have some structure, hidden from plain sight using only (linear) change of bases. These cover, for example, all schemes in the UOV family of cryptosystems [40,25] including the candidate signature schemes [17,59,16,36] in the additional round of the NIST standardization process for post-quantum cryptography [52]. Furthermore, the security of these, as well as of many other code-based and lattice-based schemes can be reduced to solving a structured systems obtained by *algebraic modeling* of the cryptosystems [2,31,37,7,54].

For these structure systems we can still use the above-mentioned algorithms, however there are two major caveats: First, the analysis for generic systems will likely be very unsuitable for predicting their run time, and second, based on the structure of the polynomial systems, these algorithms might perform many unnecessary operations. Thus, naturally, we would like to both provide a tight analysis and improve the efficiency.

A common type of polynomial systems that occurs very often in multivariate cryptanalysis are bi-homogeneous, or more generally, multi-homogeneous polynomials. For example, bi-homogeneous polynomials contain only monomials in two disjoint set of variables where the variables from each of the sets are homogeneous at some fixed degree, i.e. we say that they have some fixed

³ There exist other types of algorithms for the PoSSo problem like the probabilistic method by Lokshtanov et al. [46,18,27] with comparable time complexity to algebraic methods and SAT solvers [47,6,4] suitable only for very sparse systems over the binary field.

bi-degree (d_1, d_2) . When we have multiple such sets, we are talking about multi-homogeneous polynomials of some multi-degree $(d_1, d_2, \dots, d_c)$. Often the multi-homogeneous structure is a more fine-grained refinement of a bi-homogeneous polynomial structure.

Such types of polynomials appear for example in the Kipnis-Shamir (KS) model of MinRank [41]. The KS model further underlies equivalent key and good key attacks against step-wise multivariate schemes [55] such as Rainbow, and in particular the Rainbow-band separation attack [26]. Other examples include the reconciliation attack against UOV schemes [26,15,20] and algebraic modelings of MCE (matrix code equivalence) and ATFE (alternating trilinear form equivalence) [54].

In this paper, we focus on these multi-homogeneous polynomial systems both from a theoretical and practical perspective.

1.1 Related work

The treatment of multi-homogeneous polynomial systems and the design of dedicated version of Gröbner bases algorithms dates at least back to the work of Kreuzer and Robiano [43] that investigated the Buchberger’s algorithm for multi-graded rings. A first systematic analysis for bi-homogeneous systems was provided by Faugere et al. [34] where the authors prove an explicit form for the Hilbert series of generic bi-homogeneous systems and provide a dedicated bi-homogeneous version of the F5 algorithm. This paper upper bounds the solving degree as $d < \min(n_1, n_2) + 1$ where n_1, n_2 are the sizes of the two sets of variables.

This analysis was significantly improved by Verbel et al. [58], which specifically treats polynomials systems from the KS model. They show that these systems are not generic bilinear and identify non-trivial structural degree falls that are a result of how these systems are constructed. Making use of the specific linear part of the KS equations that only contains the linear combination variables, the authors show the correctness of a dedicated, more efficient XL algorithm in which the polynomials are only multiplied by kernel variables, named **yXL**. They do not however, use the additional multi-homogeneous structure to further improve the analysis nor heuristically, in the described dedicated **yXL** algorithm.

Further improvements in the analysis was provided by Perlner and Smith-Tone [53] who analyze systems arising from the Rainbow band separation attack [26] and consider them as a collection of both generic quadratic and bi-homogeneous polynomials (the latter being basically the KS polynomials). More general treatment of the KS equations was initiated by Nakamura et al. [51] but it lacks rigorous theoretical analysis.

1.2 Our contribution

In this paper we focus on multi-homogeneous systems both from theoretical and practical perspective. With the goal of minimizing the cost of solving such

systems, the main idea is to only construct the part of the Macaulay matrix that is “used” by multi-homogeneous polynomials. Thus, we take a more fine-grained approach and instead of proving statements for the full Macaulay matrix at some degree D , we consider a solving degree in the form of a multi-degree and further as a set of multi-degrees. This is a direct generalization of the bi-homogeneous case treated in the prior literature. We emphasize that our results hold for a general multi-homogeneous case where the given polynomials can all have a different multi-degree.

First, we provide an in-depth analysis of this case and extend the theory of semi-regular sequences to the case of multi-homogeneous ideals. We introduce several new notions that aid our theory. The first one is “border-regularity” which is a generalization for multi-homogeneous systems of the well-known notion of semi-regularity in the single-graded case. Unlike the single-graded case, it is not immediately obvious from the Hilbert series of border-regular systems which multi-degree should be taken as the solving-multi-degree, or even that taking a single multi-degree is optimal. For this, we introduce the notion of “gluability” to be able to non-ambiguously determine the best set of multi-degrees at which the system can be solved. We then prove exactly the nullity of Macaulay matrices, which gives us an exact statement about the size of the Macaulay matrix we need for solving the system.

Second, we use these theoretical results to devise a dedicated XL-type algorithm, that we call Multi-homogeneous XL, specifically optimized for solving multi-homogeneous polynomial systems. Our theory proves the correctness of the algorithm and provides an explicit formula for its time and memory complexity.

Third, we apply our theory to predict the time and memory complexity of solving systems that are regularly encountered in cryptographic literature. We consider two types of systems - the Kipnis-Shamir model for solving MinRank and the reconciliation attack on UOV. We show that our algorithm can be used with significantly decreased matrix sizes compared to regular XL.

Finally, we provide a parallelized C implementation of our multi-homogeneous XL optimized with AVX2 and AVX512 instructions, which uses the block Lanczos algorithm as a sparse linear algebra subroutine for finding kernel vectors of the Macaulay matrix. We further applied the bitslicing technique to improve efficiency and reduce branch misprediction. Our implementation supports multi-homogeneous systems of arbitrary multi-degrees and can glue them together to construct Macaulay matrices of combined multi-degrees. We provide experimental results that demonstrate the performance of our implementation and the advantages of our multi-homogeneous XL. Our implementation is available at

<https://github.com/kcning/MinRankSolver>

2 Preliminaries

Throughout this text, we will denote by $\mathbb{F}_q$ the finite field of q elements, and by $\mathbb{F}_q[x_1, \dots, x_n]$ the ring of polynomials in the variables $x_1, \dots, x_n$ over $\mathbb{F}_q$. We

use bold letters $\mathbf{a}, \mathbf{c}, \mathbf{x}, \dots$ to denote vectors over different domains. The entries of a vector $\mathbf{a}$ will be denoted by a_i . By $\text{coeff}(u, f)$ we denote the coefficient of the monomial u in the polynomial f .

2.1 Macaulay matrices

Let $\mathcal{F} = (f_1, \dots, f_m) \in \mathbb{F}_q^m[x_1, \dots, x_n]$ be a system of polynomials. We are interested in the ideal generated by $\mathcal{F}$

$$\mathcal{I}(\mathcal{F}) = \langle f_1, \dots, f_m \rangle = \text{Span}_{\mathbb{F}_q} \{u \cdot f \mid u \in \mathbb{F}_q[x_1, \dots, x_n], f \in \mathcal{F}\}$$

and its restriction up to a given degree d :

$$\mathcal{I}(\mathcal{F})_{\leq d} := \langle f_1, \dots, f_m \rangle_{\leq d} = \text{Span}_{\mathbb{F}_q} \{u \cdot f \mid u \in \mathbb{F}_q[x_1, \dots, x_n], f \in \mathcal{F}, \deg(uf) \leq d\}.$$

A *syzygy* for $\mathcal{F} = (f_1, \dots, f_m) \in \mathbb{F}_q^m[x_1, \dots, x_n]$ is an element $(s_1, \dots, s_m) \in \mathbb{F}_q^m[x_1, \dots, x_n]$ such that $\sum_{i=1}^m f_i s_i = 0$. The degree of the syzygy is defined as $\deg(\sum_{i=1}^m f_i s_i)$. The set of all syzygies forms a submodule in $\mathbb{F}_q^m[x_1, \dots, x_n]$ called the *syzygy module*.

The algorithmic counterpart of $\mathcal{I}(\mathcal{F})_{\leq d}$ will be the degree d Macaulay matrix of $\mathcal{F}$ that we denote by $\mathcal{M}_d(\mathcal{F})$. $\mathcal{M}_d(\mathcal{F})$ has columns indexed by the set of all monomials from $\mathbb{F}_q[x_1, \dots, x_n]$ up to degree d , and rows indexed by uf_i where u are all monomials such that $\deg(uf_i) \leq d$. The entries of $\mathcal{M}_d(\mathcal{F})$ are the coefficients of uf_i .

It can be seen that the row-space of $\mathcal{M}_d(\mathcal{F})$ is isomorphic to $\mathcal{I}(\mathcal{F})_{\leq d}$ while all the operations on $\mathcal{M}_d(\mathcal{F})$ boil down to simple linear algebra over $\mathbb{F}_q$.

Let $p(t)$ be a power series in t . We denote by $[t^n]p(t)$ the coefficient of t^n in the series $p(t)$. Then (see for ex. [62]), the number of columns $N((\mathcal{M}_d(\mathcal{F})))$ of the Macaulay matrix $\mathcal{M}_d(\mathcal{F})$ is $[t^d](\frac{(1-t^q)^n}{(1-t)^{n+1}})$, and the number of rows is $m \cdot [t^{d-2}](\frac{(1-t^q)^n}{(1-t)^{n+1}})$.

2.2 Polynomial system solvers and the XL algorithm

The introduction of the Macaulay matrix already gives a hint on how we might attempt at solving a given polynomial system $\mathcal{F} = (f_1, \dots, f_m) \in \mathbb{F}_q[x_1, \dots, x_n]$. Indeed, implicitly or explicitly, the Macaulay matrix is at the core of various Gröbner basis algorithms [19,45,29,24,10,39]. These algorithms often have different descriptions and efficiency considerations, but they share the same underlying idea. The idea is to find elements in the ideal generated by $\mathcal{F}$ from which the solution can be easily extracted. These are typically linear or univariate polynomials. Thus the procedure would necessarily include techniques for finding algebraic combinations of ideal elements in which most of the terms cancel out. This can, for example, be done by performing linear algebra on the Macaulay matrix of some degree d . If the matrix contains enough independent rows, i.e. if

$$\text{Rank}(\mathcal{M}_d(\mathcal{F})) \geq N((\mathcal{M}_d(\mathcal{F}))) - c \quad (1)$$

for a small constant c , we can efficiently extract the solution from the reduced echelon form of the matrix.

This is the essence of the XL (extended linearization) algorithm [45,24]. A more detailed description can be found in Algorithm 1. In the algorithm, let $\mathbf{y}$ denote the column vector of $\binom{n+d}{d}$ variables, that correspond to the columns of $\mathcal{M}_d$.

Algorithm 1 The XL algorithm [45,24]

Input:

A polynomial system $\mathcal{F} = (f_1, \dots, f_m) \in \mathbb{F}_q[x_1, \dots, x_n]$
A degree d such that (1) is satisfied

Compute the degree d Macaulay matrix $\mathcal{M}_d$

Solve the system $\mathcal{M}_d \cdot \mathbf{y} = 0$ and extract the solution $(u_1, \dots, u_n)$ for $(x_1, \dots, x_n)$

Output: $(u_1, \dots, u_n)$ s.t. $\mathcal{F}(u_1, \dots, u_n) = 0$

Generally, a major hurdle is to estimate at which degree d we need to form the Macaulay matrix $\mathcal{M}_d(\mathcal{F})$ in order to have (1) satisfied, since not all rows will be linearly independent. However, heuristically, we can make some approximations based on assumptions about (structured) random systems. For example, we could assume that the only dependent rows in the Macaulay matrix come from trivial relations, i.e. syzygies $f_i f_j - f_j f_i = 0$ that can't be avoided. Systems that have only these types of syzygies are often called semi-regular systems [9], and for these we have a pretty good understanding of the required d . More formally we call the system *semi-regular* if its Hilbert series is given by

$$\mathcal{H}(t) = \left[\frac{\prod_{i=1}^m (1 - t^{\deg f_i})}{(1 - t)^n} \right]_+.$$

Once we know the operating degree d of the algorithm, what remains is to solve the system $\mathcal{M}_d \cdot \mathbf{y} = 0$. Basic Gaussian elimination could work, but we can actually use sparse linear algebra solvers such as Block-Wiedemann [60,23] or Block-Lanczos [49,57], since all rows of the Macaulay matrix contain exactly as many non-zero terms as each of the initial polynomials. Using these, we can solve the system in time approximately:

$$c\rho \binom{n-1+d}{d}^2$$

where ρ is the density, i.e. the non-zero terms in each row, and c is a small constant.

2.3 Multi-homogeneous polynomial systems

Let $n_1, \dots, n_c \in \mathbb{N}$ and let $\mathbf{x}_i = \{x_{i1}, \dots, x_{in_i}\}$, for all $i \in \{1, \dots, c\}$. Let $\mathbb{F}_q[\mathbf{x}_i]_{d_i}$ denote the space of all homogeneous polynomials in the variables $\mathbf{x}_i$ at degree

d_i . Let

$$\mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c]_{\mathbf{d}} = \bigotimes_{i=1}^c \mathbb{F}_q[\mathbf{x}_i]_{d_i}$$

denote the space of multi-homogeneous polynomials in $\mathbf{x}_1, \dots, \mathbf{x}_c$ at degrees $d_1, \dots, d_c$ respectively, where $\mathbf{d} = (d_1, \dots, d_c) \in \mathbb{Z}_{\geq 0}^c$. This makes

$$\mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c] = \bigoplus_{\mathbf{d} \in \mathbb{Z}_{\geq 0}^c} \mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c]_{\mathbf{d}}$$

a $\mathbb{Z}_{\geq 0}^c$ -graded polynomial ring. We will use the following shorthand notation $\mathcal{R}_{\mathbf{d}} = \mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c]_{\mathbf{d}}$ and $\mathcal{R}_{\preceq \mathbf{d}} = \bigoplus_{\mathbf{d}' \preceq \mathbf{d}} \mathcal{R}_{\mathbf{d}'}$.

We order the multi-grading partially by defining $\mathbf{d} \preceq \mathbf{d}'$ iff $d_i \leq d'_i$ for all $i \in \{1, \dots, c\}$. By $\mathbf{e}_i$ for $1 \leq i \leq c$ we will denote the multi-degree $(0, \dots, 0, 1, 0, \dots, 0)$ having a 1 at position i .

Definition 1. *The multi-degree of a polynomial $f \in \mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c]$ is given by*

$$\deg(f) = (\deg_{\mathbf{x}_1}(f), \dots, \deg_{\mathbf{x}_c}(f)).$$

where $\deg_{\mathbf{x}_i}(f)$ denotes the degree of $\mathbf{x}_i$ in f . The shape of a system of polynomials $(f_1, \dots, f_m)$ is given by

$$(\deg(f_1), \dots, \deg(f_m)) \in (\mathbb{Z}_{\geq 0}^c)^m$$

Remark 1. Note that, counterintuitively, an inhomogeneous polynomial of multi-degree $\mathbf{d}$ does not necessarily contain monomials of multi-degree $\mathbf{d}$. For example, $f = x_1 + y_1$ in variable sets $\mathbf{x}, \mathbf{y}$ is of multi-degree $(1, 1)$ but has no bilinear monomials.

2.4 The MinRank problem and the Kipnis-Shamir modeling

Problem 1. Let n, m, k, r be natural numbers such that $r \leq \min(n, m)$. Given matrices $M_0, \dots, M_k \in \mathbb{F}_q^{n \times m}$, the MinRank problem $MR(n, m, k, r)$ asks us to find $\lambda_1, \dots, \lambda_k$ such that:

$$\text{Rk} \left(M_0 + \sum_{i=1}^k \lambda_i M_i \right) \leq r.$$

For a vector $\lambda = (\lambda_1, \dots, \lambda_k)$ and matrices $M_0, \dots, M_k$ let us denote $M_{\lambda} = M_0 + \sum_{i=1}^k \lambda_i M_i$. Then, for λ to satisfy the MinRank problem, we know that M_{λ} has a $(n - r)$ -dimensional left kernel V . The Kipnis-Shamir parametrizes both λ and the kernel V to build a multivariate polynomial system. Note first, that

with very high probability, V is generated by the rows of⁴:

$$\begin{pmatrix} & v_{11} & \dots & v_{1r} \\ \mathbf{I}_{n-r} & \vdots & \ddots & \vdots \\ & v_{(n-r)1} & \dots & v_{(n-r)r} \end{pmatrix}$$

This leads us to the following model, proposed by Kipnis and Shamir [41].

Model 1 *Given a $MR(n, m, k, r)$ problem with matrices $M_0, \dots, M_k \in \mathbb{F}_q^{n \times m}$. Fix a $1 \leq c \leq n - r$. The following polynomials in $\mathbb{F}_q[\lambda_1, \dots, \lambda_k, x_{11}, \dots, x_{cr}]$ define the KS-system:*

$$\begin{pmatrix} & x_{11} & \dots & x_{1r} \\ \mathbf{I}_c & \mathbf{0}_{c \times (n-r-c)} & \vdots & \ddots & \vdots \\ & & x_{c1} & \dots & x_{cr} \end{pmatrix} \cdot M_\lambda = \mathbf{0}_{c \times m}. \quad (2)$$

These polynomials are bilinear in the variable sets $\{x_{11}, \dots, x_{cr}\}$ and $\{\lambda_1, \dots, \lambda_k\}$. In fact, for each $1 \leq i \leq c$, we get m equations that are bilinear in the sets $\{x_{i1}, \dots, x_{ir}\}$ and $\{\lambda_1, \dots, \lambda_k\}$.

If we take a closer look at the equations in (2) we see that not only are these polynomials bilinear in the variable sets $\{x_{11}, \dots, x_{cr}\}$ and $\{\lambda_1, \dots, \lambda_k\}$, but in fact, for each $1 \leq i \leq c$, we get m equations that are bilinear in the sets $\{x_{i1}, \dots, x_{ir}\}$ and $\{\lambda_1, \dots, \lambda_k\}$. Then, this consists of polynomials of the form $f_i(\lambda, \mathbf{x}_1), \dots, h_i(\lambda, \mathbf{x}_c)$, for $1 \leq i \leq m$, i.e. the system is multi-homogeneous.

From the definition, we can deduce that a solution to the MinRank problem indeed leads to a solution of the polynomial system. Whether any solution to this polynomial system leads to a solution of the MinRank problem depends highly on the choice of c and is likely whenever $k < c(m - n + c)$. Nevertheless, when $c = n - r$ we are sure that any solution to the system is a solution to our MinRank problem as well.

Before going further, let us note that there exist other algebraic modelings of the MinRank problem such as the Minors modeling [33,35] and the Support Minors modeling [8], but in this paper we will be mostly interested in the Kipnis-Shamir modeling because of the multi-homogeneous structure of the modeling.

2.5 UOV and the reconciliation attack model

Another interesting example of a multi-homogeneous structure in the algebraic model of an attack occurs in the reconciliation attack on UOV [26]. The multi-homogeneous structure of this modeling was first uncovered in [20] for SNOVA.

⁴ Note that with a low probability a randomization in a form of a permutation of the columns of the kernel vectors is necessary (likely only once), in case no bases of the kernel can be written in this form. From now on we assume that this basis has this form.

Example 1. In the reconciliation attack we have the following type of equations:

$$\begin{aligned} p_i(\mathbf{o}_i) &= 0 \\ p'_{i,j}(\mathbf{o}_i, \mathbf{o}_j) &= 0 \end{aligned} \quad \forall 1 \leq i < j \leq o. \quad (3)$$

for unknown vectors $\mathbf{o}_i, 1 \leq i < j \leq o$, where p_i is quadratic and $p'_{i,j}$ is bilinear. So, we can consider the polynomials as multi-homogeneous in $\mathbb{F}_q[\mathbf{o}_1, \dots, \mathbf{o}_o]$ where

$$\begin{aligned} \deg(p_i) &= (0, \dots, 1_i, \dots, 0) \text{ and} \\ \deg(p_{i,j}) &= (0, \dots, 1_i, \dots, 1_j, \dots, 0) \quad \forall 1 \leq i < j \leq o \end{aligned}$$

To put the equations in context, $p = (p_1, \dots, p_m) : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$ is the set of public multivariate quadratic map of UOV, and p' is the polar form of p defined as

$$p'(\mathbf{u}, \mathbf{v}) = p(\mathbf{u} + \mathbf{v}) - p(\mathbf{u}) - p(\mathbf{v}), \quad \forall \mathbf{u}, \mathbf{v} \in \mathbb{F}_q^n.$$

By definition [15], the secret key of UOV is a secret subspace of $\mathbb{F}_q^n$ of dimension o that vanishes on the equations (3). Thus solving (3) breaks UOV. Typically, in (3), we only need to consider a small number c of oil vectors and not a full basis of the oil space.

For the lifted reconciliation attack [50] against SNOVA [59] a similar system arises. However, in this case the polar forms are not constructed from quadratic polynomials and hence not symmetric. Therefore we get different equations for $p'(\mathbf{o}_i, \mathbf{o}_j)$ and $p'(\mathbf{o}_j, \mathbf{o}_i)$. Thus we have $2m$ equations in degree $\mathbf{e}_i + \mathbf{e}_j$ instead of m if $i \neq j$.

3 Exploiting the multi-homogeneous structure to improve efficiency

The main idea of our work is to use this multi-graded structure in our XL-like algorithm. Previous approaches for solving the KS system used the structure for estimating the total solving degree [34], the first fall degree [58], or partially for improving the XL algorithm by multiplying only with kernel variables $x_1, \dots, x_r, y_1, \dots, y_r, z_1, \dots, z_r$ [58].

Here, we consider our solving degree $\mathbf{d}$ to be instead in the form of a multi-degree $\mathbf{d} = (d_\lambda, d_{x_1}, \dots, d_{x_c})$ or a set of multi-degrees $\mathcal{D}$. To aid the reading process, let us first consider the simpler case of a single multi-degree $\mathbf{d}$. What this means in practice is that we consider Macaulay matrices where our columns are now indexed by monomials with multi-degree at most $\mathbf{d}$. Then the rows are exactly these products of monomials and polynomials that can be expressed by such monomials.

Example 2. Another look at the Kipnis-Shamir modeling for MinRank
Let us fix $c = 3$ so that we are working in the polynomial ring

$$\mathbb{F}_q[\lambda_1, \dots, \lambda_k, x_1, \dots, x_r, y_1, \dots, y_r, z_1, \dots, z_r].$$

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & x_1 & \dots & x_r \\ 0 & 1 & 0 & 0 & \dots & 0 & y_1 & \dots & y_r \\ 0 & 0 & 1 & 0 & 0 & 0 & z_1 & \dots & z_r \end{pmatrix} \cdot \left(M_0 + \sum_{i=1}^k \lambda_i M_i \right) = \mathbf{0}_{3 \times m}. \quad (4)$$

Then, this consists of polynomials $f_i(\lambda, x)$, $g_i(\lambda, y)$, and $h_i(\lambda, z)$ for $1 \leq i \leq m$, i.e. the system is multi-homogeneous.

Let us consider building the Macaulay matrix of the system (4) in multi-degree $(d_\lambda, d_x, d_y, d_z) = (2, 3, 2, 1)$. Then our columns are exactly the monomials

$$\lambda_{i_1} \lambda_{i_2} x_{j_1} x_{j_2} x_{j_3} y_{k_1} y_{k_2} z_{l_1}.$$

Here, each factor may be 1 if we are in the inhomogeneous case. Furthermore, our rows are now given by the polynomials

$$\begin{aligned} & \lambda_{i_1} x_{j_1} x_{j_2} y_{k_1} y_{k_2} z_{l_1} \cdot f_i(\lambda, x), \\ & \lambda_{i_1} x_{j_1} x_{j_2} x_{j_3} y_{k_1} z_{l_1} \cdot g_i(\lambda, y), \\ & \lambda_{i_1} x_{j_1} x_{j_2} x_{j_3} y_{k_1} y_{k_2} \cdot h_i(\lambda, z). \end{aligned}$$

Then since each of our polynomials is bilinear we know that our products can be described entirely in the Macaulay matrix with the given columns. Here we can also see the difference with regular XL. Namely, that we do not consider all (monomial, polynomial) product pairs, which reduces the size of the matrix we work with, and thus the complexity of solving the system.

3.1 Multi-degrees and building a multi-degree Macaulay matrix

In this subsection, we formalize the notions implied in Example 2. Just as in the standard grading when $c = 1$, we can build the multi-degree-restricted ideal and the Macaulay matrix at a multi-degree $\mathbf{d}$.

Definition 2. Let $\mathcal{F} = (f_1, \dots, f_m) \in \mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c]$ be a system of polynomials and $\mathbf{d} \in \mathbb{Z}_{\geq 0}^c$ a multi-degree. Then we define the $\mathbf{d}$ -restricted ideal of $\mathcal{F}$ to be

$$\begin{aligned} \mathcal{I}(\mathcal{F})_{\preceq \mathbf{d}} &:= \langle f_1, \dots, f_m \rangle_{\preceq \mathbf{d}} \\ &= \text{Span}_{\mathbb{F}_q} \{ u \cdot f \mid u \in \mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c], f \in \mathcal{F}, \deg(u f) \preceq \mathbf{d} \}. \end{aligned}$$

Also, we define the Macaulay matrix $\mathcal{M}_{\mathbf{d}}$ of $\mathcal{F}$ in multi-degree $\mathbf{d}$ to be the matrix with columns indexed by monomials u such that $\deg(u) \preceq \mathbf{d}$, rows indexed by pairs (t, f_i) where t is a monomial such that $\deg(t \cdot f_i) \preceq \mathbf{d}$, and coefficients given by $\text{coeff}(u, t \cdot f_i)$.

Proposition 1. Let $\mathcal{F} = (f_1, \dots, f_m) \in \mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c]$ be a system of polynomials and $\mathbf{d} \in \mathbb{Z}_{\geq 0}^c$ a multi-degree, then:

$$\mathcal{I}(\mathcal{F})_{\preceq \mathbf{d}} \cong \text{Row}(\mathcal{M}_{\mathbf{d}}).$$

As a first observation we can see that $\mathcal{M}_{\mathbf{d}}$ is a submatrix of the singly graded Macaulay matrix of $\mathcal{F}$ in degree $\sum_i d_i$.

Just as in the singly graded case we are interested again in the number of rows, number of columns, and the rank of the Macaulay matrix.

Proposition 2. *Let $\mathcal{F} \subset \mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c]$ be a system of polynomials and $\mathbf{d} \in \mathbb{Z}_{\geq 0}^c$ a multi-degree. The Macaulay matrix of $\mathcal{F}$ in degree $\mathbf{d}$ has*

$$\prod_{i=1}^d \binom{n_i + d_i}{d_i} \text{ columns} \quad \text{and} \quad \sum_{j=1}^m \prod_{i=1}^d \binom{n_i + d_i - \deg_{\mathbf{x}_i}(f_j)}{d_i - \deg_{\mathbf{x}_i}(f_j)} \text{ rows.}$$

Let us introduce the shorthand notation $\binom{\mathbf{a}}{\mathbf{b}} = \prod_{i=1}^c \binom{a_i}{b_i}$ for $\mathbf{a}, \mathbf{b} \in \mathbb{Z}_{\geq 0}^c$. Then the amount of columns are given simply by $\binom{\mathbf{n} + \mathbf{d}}{\mathbf{d}}$ and the amount of rows by

$$\sum_{j=1}^m \binom{\mathbf{n} + \mathbf{d} - \deg(f_j)}{\mathbf{d} - \deg(f_j)}$$

3.2 Regularity assumptions

In the algorithm we are interested in Macaulay matrices whose rank defect, or right nullity, is equal to 1. For predicting the rank of these multi-graded Macaulay matrices we need to have an analysis of the trivial syzygies that appear. Furthermore, we need to assume that in the generic case no additional syzygies appear.

The problem with this however, is that, due to Cramer's rule, additional syzygies *do* appear. This was shown in [34] for bihomogeneous systems and holds similarly for multi-homogeneous systems. The reason for this relies on the fact that if f is homogeneous of degree $d > 0$ in the $\mathbf{x}_i$ variables, we can write

$$d \cdot f = \sum_j \frac{\partial f}{\partial x_{ij}} x_{ij}$$

The Jacobian with respect to $\mathbf{x}_i$ is given by

$$\text{Jac}_{\mathbf{x}_i}(f_1, \dots, f_m) = \begin{bmatrix} \frac{\partial f_1}{\partial x_{i1}} & \dots & \frac{\partial f_1}{\partial x_{in}} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_{i1}} & \dots & \frac{\partial f_m}{\partial x_{in}} \end{bmatrix}.$$

And then, with the above observation we find

$$\text{Jac}_{\mathbf{x}_i} \cdot \begin{bmatrix} x_{11} \\ \vdots \\ x_{1n} \end{bmatrix} = \begin{bmatrix} \deg_{\mathbf{x}_i}(f_1) \cdot f_1 \\ \vdots \\ \deg_{\mathbf{x}_i}(f_m) \cdot f_m \end{bmatrix}.$$

This means that any element of the left kernel of $\text{Jac}_{\mathbf{x}_i}$, leads to a syzygy if $\deg_{\mathbf{x}_i}(f_j) \neq 0 \in \mathbb{F}_q$ for some j . When $m = n_i + 1$ we can construct such an element by Cramer's rule

$$(-\text{minor}(\text{Jac}_{\mathbf{x}_i}, 1), \dots, (-1)^m \text{minor}(\text{Jac}_{\mathbf{x}_i}, m))$$

Assuming $\deg_{\mathbf{x}_i}(f_j) \neq 0 \in \mathbb{F}_q$ for all j we can thus construct syzygies of multi-degree

$$\mathbf{e}_i + \sum_{j=1}^m (\deg(f_j) - \mathbf{e}_i).$$

By applying this to bilinear equations, i.e. of degree $(1, 1)$ we indeed obtain the same syzygies as in [34] of degree $(1, n_x)$ and $(n_y, 1)$. This can be generalized for any $m > n_i$ by applying this technique to $(n_i + 1) \times n_i$ submatrices of $\text{Jac}_{\mathbf{x}_i}$.

While it would be interesting to count these syzygies exactly for all shapes in all multi-degrees, we do not need this level of precision. Instead we are going to exclude those cases from our conjecture.

Definition 3. Let $\mathcal{S} = (\mathbf{d}_1, \dots, \mathbf{d}_m)$ be a shape of sets of polynomials in $\mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c]$. Then we call the border of the shape $\mathcal{S}$ to be the set of multi-degrees

$$\mathcal{B}(\mathcal{S}) = \left\{ -n_i \cdot \mathbf{e}_i + \sum_{\ell=1}^{n_i+1} \mathbf{d}_{j_\ell} \mid 1 \leq i \leq c, 1 \leq j_1 < \dots < j_{n_i+1} \leq m, (\mathbf{d}_{j_\ell})_i > 0 \ \forall \ell \right\}$$

In other words $\mathcal{B}(\mathcal{S})$ is the set of multi-degrees where we can construct syzygies due to Cramer's rule. Let us denote $\mathcal{B}_{\prec}(\mathcal{S}) = \{\mathbf{d} \mid \mathbf{d} \prec \mathbf{b} \ \forall \mathbf{b} \in \mathcal{B}(\mathcal{S})\}$

Definition 4. Let $\mathcal{F} = (f_1, \dots, f_m) \in \mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c]$ be a system of multi-homogeneous polynomials of shape $\mathcal{S}$. We call the system $(f_1, \dots, f_m)$ border-regular if its Hilbert series matches

$$\mathcal{H}(\mathbf{t}) = \left[\frac{\prod_{j=1}^m (1 - \mathbf{t}^{\deg f_j})}{\prod_{i=1}^c (1 - t_i)^{n_i}} \right]_+$$

on the coefficient of $\mathbf{t}^{\mathbf{d}}$ for all $\mathbf{d} \in \mathcal{B}_{\prec}(\mathcal{S})$. Here we use the shorthand notation $\mathbf{t}^{\mathbf{d}} = \prod_{i=1}^c t_i^{d_i}$.

Conjecture 1. For given shape $\mathcal{S} = (\mathbf{d}_1, \dots, \mathbf{d}_m)$, a random system $f_1, \dots, f_m \in \mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c]$ with shape $\mathcal{S}$ is border-regular with a very high probability.

Remark 2. Note the analogy with semi-regular systems in the single-graded case. The border-regular property also defines an open subset of the moduli space of systems with shape $\mathcal{S}$. Therefore, if this set is non-empty, i.e. if any such system exists, it is a generic property. Having such a generic property is nice because that means that, for infinite fields, almost all systems have this property. The expectation then is that for finite fields, the same holds and the ratio of systems that have this property grows to one as the size of the polynomial ring and the size of the shapes increase.

3.3 Admissible multi-degrees

As usual, we are interested in Macaulay matrices having corank-1. One pitfall when considering operating multi-degrees is that some multi-degrees might not contain all system polynomials. In that case finding a right kernel vector might not result in an solution to *all* equations. So for a shape $\mathcal{S} = (\mathbf{d}_1, \dots, \mathbf{d}_m)$ let us denote $\mathcal{S}^+$ to be the set of multi-degrees $\mathbf{d}$ for which $\mathbf{d}_i \preceq \mathbf{d}$ for all i . Furthermore, let us denote

$$\mathcal{H}_{\mathcal{S}}(\mathbf{t}) = \frac{\prod_{j=1}^m (1 - \mathbf{t}^{\mathbf{d}_j})}{\prod_{i=1}^c (1 - t_i)^{n_i}}$$

Definition 5. Let $\mathcal{S}$ be a shape for systems in $\mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c]$. We call a multi-degree $\mathbf{d} \in \mathcal{S}^+ \cap \mathcal{B}_{\prec}(\mathcal{S})$ admissible if

$$[\mathcal{H}_{\mathcal{S}}(\mathbf{t})]_+ [\mathbf{t}^{\mathbf{d}}] \leq 1$$

Note that, by definition of $[\cdot]_+$, if $\mathbf{d}$ is admissible, then any $\mathbf{d}' \in \mathcal{B}_{\prec}(\mathcal{S})$ with $\mathbf{d} \preceq \mathbf{d}'$ is also admissible. In contrast to regular XL, the set of admissible multi-degrees might not have a single well-defined minimum.

3.4 The inhomogeneous case, gluing Macaulay matrices

In the previous section we have argued that working with a multi-degree Macaulay matrix can be sufficient for solving the system, while providing a substantial efficiency gain. Yet, as the next example demonstrates, that is not the full story, and we can do better.

Example 3. Consider a tri-graded system with $(n_x, n_y, n_z) = (3, 3, 3)$ having 6 equations in degree $(1, 1, 0)$ and 6 equations in degree $(1, 0, 1)$. Then the admissible tri-degree with the least monomials is $(1, 1, 3)$, having a Macaulay size of $\binom{3+1}{1} \binom{3+1}{1} \binom{3+3}{3} = 320$ columns and $6 \binom{3+3}{3} + 6 \binom{3+1}{1} \binom{3+2}{2} = 360$ rows. Note that due to $d_x = 1$ there are no trivial syzygies.

Now, consider the Macaulay matrix in degree $(1, 1, 2)$. we find that, with 156 linear independent rows, we can eliminate all columns of monomials of exactly degree $(1, 1, 2)$, $(1, 0, 2)$, $(0, 1, 2)$ and $(0, 0, 2)$ and are left with 60 rows in degrees at most $(1, 1, 1)$. We can do the same in degree $(1, 2, 1)$, this gives us an entirely different Macaulay matrix! Then, by echolonizing, we again get 60 rows in degrees at most $(1, 1, 1)$. Because part of these equations come from degree falls in differing Macaulay matrix, we expect them to be independent as well. And experiments indeed verify that these two sets of 60 equations span the entire space of dimension 64.

Due to the overlap, the Macaulay matrix having monomials of degree at most $(1, 1, 2)$ or $(1, 2, 1)$ only has 256 columns and 272 rows. This is much smaller than the Macaulay matrix in admissible degree $(1, 1, 3)$.

Hence, we propose, not only to use Macaulay matrices at some multi-degree, but also to “merge together” matrices at several multi-degrees for the purpose of getting an even smaller matrix that can be used to solve the system. We call this *gluing*. To properly define such gluings we need to consider subsets of $\mathbb{Z}_{\geq 0}^c$ that are closed downwards. For this we borrow terminology from Poset theory and Border Basis theory, specifically from [44].⁵

Definition 6. Let $\mathcal{D} \subset \mathbb{Z}_{\geq 0}^c$ be a subset of multi-degrees. Then $\mathcal{D}$ is called an order ideal if $\mathbf{d} \in \mathcal{D}$ and $\mathbf{d}' \preceq \mathbf{d}$ implies $\mathbf{d}' \in \mathcal{D}$. The closure of an arbitrary subset $\mathcal{D} \subset \mathbb{Z}_{\geq 0}^c$ is the smallest order ideal containing $\mathcal{D}$. We denote the closure by $\overline{\mathcal{D}}$ or $\langle \mathbf{D}_1, \dots, \mathbf{D}_k \rangle$ for a finite set of generators. If $\mathcal{D} = \{\mathbf{D}\}$, we write $\overline{\mathbf{D}}$ for $\overline{\mathcal{D}}$.

From here on we will use capital $\mathbf{D}_i$ for the multidegree generators of order ideals to prevent confusion with the multidegrees in the shape of a system.

Definition 7. Let $\mathcal{D} \subset \mathbb{Z}_{\geq 0}^c$ be an order ideal. Its border is defined by

$$\partial\mathcal{D} = \{\mathbf{d} + \mathbf{e}_i \mid \mathbf{d} \in \mathcal{D}, 1 \leq i \leq c\} \setminus \mathcal{D}.$$

It's unique minimal set of generators is defined by

$$\text{gen}(\mathcal{D}) = \min\{\mathcal{X} \mid \overline{\mathcal{X}} = \mathcal{D}\}$$

Example 4. Let $\mathcal{T}_d$ (resp. $\mathcal{T}_{\leq d}$) be the set of multi-degrees that sum to d (resp. at most d). Then the polynomials with monomials of degrees in $\mathcal{T}_{\leq d}$ are exactly the degree d polynomials in the total degree. We have $\partial\mathcal{T}_{\leq d} = \mathcal{T}_{d+1}$ and $\text{gen}(\mathcal{T}_{\leq d}) = \mathcal{T}_d$.

Definition 8. Let $\mathcal{F} = (f_1, \dots, f_m) \in \mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c]$ be a system of polynomials and $\mathcal{D}$ an order ideal. Then we define the $\mathcal{D}$ -restricted ideal of $\mathcal{F}$ to be

$$\mathcal{I}(\mathcal{F})_{\mathcal{D}} = \text{span}_{\mathbb{F}_q}\{u \cdot f \mid u \in \mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c], f \in \mathcal{F}, \deg(uf) \in \mathcal{D}\}.$$

Also, we define the Macaulay matrix $\mathcal{M}_{\mathcal{D}}$ of $\mathcal{F}$ with gluing $\mathcal{D}$ to be the matrix with columns indexed by monomials u such that $\deg(u) \in \mathcal{D}$, rows indexed by pairs (t, f_i) where t is a monomial such that $\deg(t \cdot f_i) \in \mathcal{D}$, and coefficients given by $\text{coeff}(u, t \cdot f_i)$. Furthermore, we call the order ideal $\mathcal{D}$ admissible for $\mathcal{F}$ if $\mathcal{I}(\mathcal{F})_{\mathcal{D}} = \mathcal{I}(\mathcal{F}) \cap \mathcal{R}_{\mathcal{D}}$.

If $\mathcal{F}$ is clear from the context we will write $\mathcal{I}_{\mathbf{D}}$ and $\mathcal{I}_{\mathcal{D}}$. The similarity with definition 2 is really apparent, and indeed, if we take $\mathcal{D} = \overline{\mathbf{D}}$ we get an equivalent definition.

Proposition 3. Let $\mathcal{F} = (f_1, \dots, f_m) \in \mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c]$ be a system of polynomials and $\mathcal{D}$ an order ideal, then:

$$\mathcal{I}(\mathcal{F})_{\mathcal{D}} \cong \text{Row}(\mathcal{M}_{\mathcal{D}}).$$

⁵ See section 0.5 in [44] for alternative terminology.

Whereas in the homogeneous case we could homogenize the polynomials $f_1, \dots, f_m$ to determine the columns, rows, and rank, now, in the gluing case, it will not give us the right order ideal. Instead we consider sums over $\mathcal{D}$.

Proposition 4. *Let $\mathcal{F} \subset \mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c]$ be a system of polynomials of shape $\mathcal{S} = (\mathbf{d}_1, \dots, \mathbf{d}_m)$ and $\mathcal{D}$ an order ideal. The Macaulay matrix of $\mathcal{F}$ with gluing $\mathcal{D}$ has*

$$\sum_{\mathbf{d} \in \mathcal{D}} \binom{\mathbf{n} + \mathbf{d} - 1}{\mathbf{d}} \text{ columns} \quad \text{and} \quad \sum_{\mathbf{d} \in \mathcal{D}} \sum_{j=1}^m \binom{\mathbf{n} + \mathbf{d} - \mathbf{d}_j - 1}{\mathbf{d} - \mathbf{d}_j} \text{ rows.}$$

Now, most importantly, we are interested in the rank of such matrices. As we saw in Example 3, the interesting behavior occurs if we have degree-falls that lie in the intersection of two or more single-multi-degree cases. In that case we can merge the degree falls to obtain a bigger space. Therefore, to make rank predictions we have to make heuristic assumptions about such mergings.

Definition 9. *Let $\mathcal{F} \subset \mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c]$ be a system of polynomials. We call $\mathcal{F}$ gluable if for any multi-degree $\mathbf{D}$ and order ideal $\mathcal{D} = \langle \mathbf{D}_1, \dots, \mathbf{D}_k \rangle$ we have that either $\overline{\mathbf{D}} \cup \mathcal{D}$ is admissible or*

$$\mathcal{I}_{\mathbf{D}} \cap \mathcal{I}_{\mathcal{D}} = \mathcal{I}_{\overline{\mathbf{D}} \cap \mathcal{D}}$$

In other words, for a gluable system, if we get degree falls in multiple Macaulay matrices $\mathcal{M}_{\mathbf{D}_i}$ to the same degree $\mathbf{D}'$ then either these degree falls are linearly independent or together they span the entire space $\mathcal{I} \cap \mathcal{R}_{\mathbf{D}'}$ and the order ideal is admissible.

In the experiments we performed this was always the case (for ex. in Example 3) and we conjecture that this holds for border-regular systems with high probability.

Conjecture 2. Border-regular systems are gluable with high probability.

Theorem 1. *Let $\mathcal{F} \subset \mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c]$ be a gluable, border-regular system of polynomials of shape $\mathcal{S} = (\mathbf{d}_1, \dots, \mathbf{d}_m)$ and $\mathcal{D}$ an order ideal such that $\mathcal{S} \subseteq \mathcal{D} \subseteq \mathcal{B}_{\prec}(\mathcal{S})$. Furthermore, assume that no sub-order ideal of $\mathcal{D}$ is admissible for $\mathcal{F}$. Then, the right nullity of the matrix $\mathcal{M}_{\mathcal{D}}$ is given by*

$$\sum_{\mathbf{d} \in \mathcal{D}} \mathcal{H}_{\mathcal{S}}(\mathbf{t})[\mathbf{t}^{\mathbf{d}}].$$

Note that we are not truncating the conjectured Hilbert series in this theorem. The reason for this is that the Hilbert series above predicts the nullity of the top-multi-homogeneous part for each $\mathbf{d} \in \mathcal{D}$. However, when the nullity of this part is negative, this means that we have degree falls into the degrees directly below $\mathbf{d}$. Since these columns are in our matrix, these do not lead to forced non-trivial syzygies.

Example 5. We have been implicitly using this formula in the multi-homogeneous case. If $\mathcal{D} = \overline{\mathbf{D}}$ we obtain the following equalities

$$\begin{aligned} \sum_{\mathbf{d} \preceq \mathbf{D}} \frac{\prod_{j=1}^m (1 - \mathbf{t}^{\mathbf{d}_j})}{\prod_{i=1}^c (1 - t_i)^{n_i}} [\mathbf{t}^{\mathbf{d}}] \\ = \frac{\prod_{j=1}^m (1 - \mathbf{t}^{\mathbf{d}_j})}{\prod_{i=1}^c (1 - t_i)^{n_i}} \prod_{i=1}^c (1 + t_i + t_i^2 + \cdots + t_i^{D_i}) [\mathbf{t}^{\mathbf{D}}] \\ = \frac{\prod_{j=1}^m (1 - \mathbf{t}^{\mathbf{d}_j})}{\prod_{i=1}^c (1 - t_i)^{n_i+1}} [\mathbf{t}^{\mathbf{D}}] \end{aligned}$$

Since the Hilbert series of the (multi)homogenization exactly predicts the right nullity, $\dim \mathcal{R}_{\preceq \mathbf{D}} - \dim \mathcal{I}(\mathcal{F})_{\mathbf{D}}$, of $\mathcal{M}_{\mathbf{D}}$.

For the general case we use a similar strategy

Proof. Pick the homogeneous case as an induction basis. We will show that if this holds for non-admissible $\mathbf{D}$, $\mathcal{D}$, then it will hold for $\overline{\mathbf{D}} \cup \mathcal{D}$ as well. First note that $\mathcal{U}$ is an order ideal as well. Furthermore, $\mathcal{R}_{\mathbf{D}} \cap \mathcal{R}_{\mathcal{D}} = \mathcal{R}_{\overline{\mathbf{D}} \cap \mathcal{D}}$ and hence

$$\dim \mathcal{R}_{\overline{\mathbf{D}} \cup \mathcal{D}} = \dim \mathcal{R}_{\mathbf{D}} + \dim \mathcal{R}_{\mathcal{D}} - \dim \mathcal{R}_{\overline{\mathbf{D}} \cap \mathcal{D}}.$$

Furthermore, by gluability and the fact that $\overline{\mathbf{D}} \cup \mathcal{D}$ is not admissible we find

$$\mathcal{I}_{\mathbf{D}} \cap \mathcal{I}_{\mathcal{D}} = \mathcal{I}_{\overline{\mathbf{D}} \cap \mathcal{D}}$$

and thus

$$\dim \mathcal{I}_{\overline{\mathbf{D}} \cup \mathcal{D}} = \dim \mathcal{I}_{\mathbf{D}} + \dim \mathcal{I}_{\mathcal{D}} - \dim \mathcal{I}_{\overline{\mathbf{D}} \cap \mathcal{D}}.$$

Then, by induction of the amount of generators we find that the given sum is the nullity

$$\dim \mathcal{R}_{\mathcal{D}} - \dim \mathcal{I}_{\mathcal{D}}$$

for an arbitrary order ideal $\mathcal{D}$.

Definition 10. Let $\mathcal{S} = (\mathbf{d}_1, \dots, \mathbf{d}_m)$ be a shape for systems in $\mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c]$. We call an order ideal $\mathcal{S} \subseteq \mathcal{D} \subseteq \mathcal{B}_{\prec}(\mathcal{S})$ admissible for $\mathcal{S}$ if

$$\sum_{\mathbf{d} \in \mathcal{D}} \frac{\prod_{j=1}^m (1 - \mathbf{t}^{\mathbf{d}_j})}{\prod_{i=1}^c (1 - t_i)^{n_i}} [\mathbf{t}^{\mathbf{d}}] \leq 1.$$

Remark 3. Note that an order ideal is admissible for a system of polynomials if the Macaulay matrix of that specific system solves in that gluing. An order ideal is admissible for a shape of systems if it is admissible for gluable border-regular systems of that shape.

4 The multi-homogeneous XL algorithm

We are now ready to define an algorithm to solve systems of multi-graded equations with an algorithm we call multi-homogeneous XL. The algorithm works for arbitrary systems of polynomials given an admissible order ideal $\mathcal{D}$. For border-regular systems, we can predict the admissible order ideals and hence the complexity of the algorithm.

Theorem 2. *Let $f_1, \dots, f_m \in \mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c]$ be a border-regular system of shape $\mathcal{S} = (\mathbf{d}_1, \dots, \mathbf{d}_m)$ and $|\mathbf{x}_i| = n_i$. Let $\mathcal{A}$ be the set of admissible order ideals for $\mathcal{S}$. Then the cost of the algorithm is*

$$\min_{\mathcal{D} \in \mathcal{A}} 2 \cdot \rho \cdot \left(\sum_{\mathbf{d} \in \mathcal{D}} \binom{\mathbf{n} + \mathbf{d} - 1}{\mathbf{d}} \right)^2.$$

Here ρ is the maximal amount of terms in a single f_i .

Proof. This follows directly from theorem 1 and applying sparse linear algebra techniques, described in section 5.2, on the Macaulay matrix with gluing $\mathcal{D}$.

Algorithm 2 Multi-homogeneous XL for a degree set $\mathcal{D}$

Input:

A border-regular set of polynomials $f_1, \dots, f_m \in \mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c]$ of shape $\mathcal{S}$
 An admissible order ideal $\mathcal{D}$ for $\mathcal{S}$

Compute the multi-degree Macaulay matrix $\mathcal{M}_{\mathcal{D}}$

$\text{Ker}(\mathcal{M}) \leftarrow \mathbf{FindLeftKernelVectors}(\mathcal{M}_{\mathcal{D}}^T, 1) \triangleright \text{Find at most 1 right kernel vector}$

Compute the solution from $\text{Ker}(\mathcal{M})$ and return the solution

By Example 4 we see that if a multi-graded structure is present, our new multi-homogeneous XL algorithm is at least as good as regular XL. The reason is that if a single-graded system solves in operating degree D , then $\mathcal{T}_D$ is an admissible order ideal. The size of the regular Macaulay matrix in degree D and the Macaulay matrix with gluing $\mathcal{T}_D$ are exactly the same. In fact, they are the same matrix. Thus, considering multi-homogeneous XL can only improve the complexity.

Remark 4. Recall that in Buchberger's algorithm [3] one finds a Gröbner basis by iteratively adding S-polynomials to the system. In essence, Buchberger's algorithm already uses the multi-grading implicitly. Given polynomials f, g we find that $\deg(\mathcal{S}(f, g)) \preceq \deg(f) + \deg(g)$. Therefore, during this step, Buchberger's algorithm is already limited to multi-degree $\deg(f) + \deg(g)$ instead of total degree $\sum_i \deg_{\mathbf{x}_i}(f) + \deg_{\mathbf{x}_i}(g)$. So in a sense we are trying to bridge the gap between XL and Buchberger regarding this structure.

4.1 Finding optimal gluings

The previous result guarantees that there are (potentially multiple) optimal sets of admissible multi-degrees, does not give an efficient method of finding these however. Indeed, this is a mayor hurdle, that currently we are not able to answer precisely. Instead we resort to a heuristic and we argue that while not guaranteed to give optimal results, it is fairly close.

Let us first note that finding the optimal order ideal generated by a single multidegree is quite straightforward and somewhat efficient. To do this we use a search algorithm in the same spirit as Dijkstra's algorithm. Since the matrix size is an increasing function with respect to the multidegree order, we can start at the origin. Then at each step, we will check whether the homogeneous $\mathcal{H}_S(\mathbf{t})$ has a non-positive coefficient for $\mathbf{d}$ where $\mathbf{d}$ is a neighbor of the multidegrees we considered so far with the lowest amount of monomials. This way we are guaranteed to find the admissible multidegree with the lowest amount of monomials. This gives us an upper bound on the size of the optimal gluing.

Then, observe that for an optimal admissible $\mathcal{D} = \langle \mathbf{D}_1, \dots, \mathbf{D}_k \rangle$ order ideal it should hold that $\mathcal{H}_S[\mathbf{t}^{\mathbf{D}_i}] < 0$ for each i . Otherwise, the same gluing without $\mathbf{D}_i$ would be an admissible gluing of smaller size. Therefore, we can list all multidegrees for which $\mathcal{H}_S(\mathbf{t})$ is negative and their homogeneous size is lower than that of the optimal single multidegree. Then clearly, the optimal $\mathcal{D}$ is generated by some subset of those.

To reduce the list of candidate generators even further we can generalize the above approach. Let $\mathbf{D}_1, \dots, \mathbf{D}_k$ be the list of candidate generators and let $\mathcal{D} = \langle \mathbf{D}_{i_1}, \dots, \mathbf{D}_{i_\ell} \rangle$ be the optimal admissible order ideal. We can extend the discussion above by claiming $\{\mathbf{D}_1, \dots, \mathbf{D}_k\} \setminus \{\mathbf{D}_1, \dots, \mathbf{D}_k\} \setminus \mathbf{D}_i$ should have a negative value for each i because otherwise it will not be part of the optimal gluing. And indeed, if $\mathbf{D}_i \in \mathcal{D}$ then $\langle \mathbf{D}_{i_1}, \dots, \mathbf{D}_{i_\ell} \rangle \setminus \{\mathbf{D}_{i_1}, \dots, \mathbf{D}_{i_\ell}\} \setminus \mathbf{D}_i \supset \langle \mathbf{D}_1, \dots, \mathbf{D}_k \rangle \setminus \{\mathbf{D}_1, \dots, \mathbf{D}_k\} \setminus \mathbf{D}_i$. Since $\mathcal{H}_S(\mathbf{t})$ for non-generators is positive, this means that $\langle \mathbf{D}_{i_1}, \dots, \mathbf{D}_{i_\ell} \rangle \setminus \{\mathbf{D}_{i_1}, \dots, \mathbf{D}_{i_\ell}\} \setminus \mathbf{D}_i$ has positive nullity, which is a contradiction with it being an optimal gluing.

If we now assume that we have a list of generators in which no $\mathbf{D}_i$ can be shown to be redundant in the above way we can generalize this to sets of 2 generators. If $\{\mathbf{D}_1, \dots, \mathbf{D}_k\} \setminus \{\mathbf{D}_1, \dots, \mathbf{D}_k\} \setminus \mathbf{D}_i$ and $\{\mathbf{D}_1, \dots, \mathbf{D}_k\} \setminus \{\mathbf{D}_1, \dots, \mathbf{D}_k\} \setminus \mathbf{D}_j$ have negative nullity and $\langle \mathbf{D}_1, \dots, \mathbf{D}_k \rangle \setminus \{\mathbf{D}_1, \dots, \mathbf{D}_k\} \setminus \{\mathbf{D}_i, \mathbf{D}_j\}$ has non-negative nullity then it follows that $\langle \mathbf{D}_{i_1}, \dots, \mathbf{D}_{i_\ell} \rangle \setminus \{\mathbf{D}_{i_1}, \dots, \mathbf{D}_{i_\ell}\} \setminus \{\mathbf{D}_i, \mathbf{D}_j\}$ must have a non-negative value as well if $\mathbf{D}_i \in \mathcal{D}$ or $\mathbf{D}_j \in \mathcal{D}$. The reason for this is that this implies $\{\mathbf{D}_1, \dots, \mathbf{D}_k\} \setminus \mathbf{D}_j \setminus \{\mathbf{D}_1, \dots, \mathbf{D}_k\} \setminus \{\mathbf{D}_i, \mathbf{D}_j\}$ must have non-negative nullity as well. Hence, in that case, we can remove both $\mathbf{D}_i$ and $\mathbf{D}_j$ from our candidate generators. After having checked all 2-element sets we can repeat this for 3-element sets and higher.

However, this reduction can be quite expensive. It forces us to first check all 1-element subsets, then all 2-elements subsets, and so forth. Still, by checking up to all 3-element subsets, we often managed to reduce the amount of candidate generators by one third. What is even more, when we do the exhaustive search

afterwards, we can distinguish the cases $\mathbf{D}_1 \subset \mathcal{D}$ and $\mathbf{D}_1 \not\subset \mathcal{D}$ and reduce $\mathbf{D}_2, \dots, \mathbf{D}_k$ in the same way.

Finally, in most cases there is symmetry present in the shapes of the equations. Let σ be a permutation of the coefficients of a multidegree. Then if $\mathcal{S} = (\mathbf{d}_1, \dots, \mathbf{d}_m)$ and $\mathcal{S}' = (\sigma(\mathbf{d}_1), \dots, \sigma(\mathbf{d}_m))$ are equal up to ordering, this must mean that our admissible order ideals are invariant under σ as well. We use this to reduce the amount of $\mathcal{H}_{\mathcal{S}}(\mathbf{t})$ computations and to reduce the exhaustive search. Heuristically, we also do the reduction described above only on the orbits.

Finally we report some examples of parameter sets where we found gluing was more optimal. The shapes of these systems are described in the following sections. In Table 1 we consider two types of parameter sets that correspond to the cases from Table 2 and 3.

Table 1. Examples of shapes for which gluing multiple generators is optimal

Systems of Kipnis-Shamir shape (r, k, m, n)							
r	k	m	n	Single generator		Best gluing	
				Degree	Size	Degrees	Size
3	3	6	5	(1, 1, 3)	320	$\{(1, 2, 1), (1, 1, 2)\}$	256
3	4	6	5	(1, 2, 3)	1000	$\{(1, 2, 2), (2, 1, 2)\}$	900
3	7	6	6	(1, 2, 2, 4)	28000	$\{(1, 3, 2, 2), (1, 2, 2, 3)\}$	24000
4	10	8	7	(1, 2, 4, 5)	1455300	$\{(1, 3, 3, 4), (1, 4, 3, 3)\}$	1414875
Systems of Reconciliation shape (v, m, o)							
v	m	o	Single generator		Best gluing		Size
			Degree	Size	Degrees	Size	
8	3	3	(3, 3, 6)	81756675	$\{(5, 3, 3), (4, 2, 4), (3, 3, 4)\}$	48923325	
8	4	4	(1, 1, 2, 3)	601425	$\{(1, 2, 1, 2), (0, 2, 2, 2), (1, 2, 2, 1), (1, 1, 2, 2)\}$	446877	

4.2 Comparing multihomogeneous XL with regular and bilinear XL

We apply our theoretical estimates to the Kipnis-Shamir model and to the Reconciliation attack model to show the improvement over regular XL.

The Kipnis-Shamir model as a multi-homogeneous system In Table 2 we can see the benefits of treating the Kipnis-Shamir modeling as a multi-homogeneous system and using our dedicated algorithm for solving it. Note that the table contains results for systems of KS shape, and not for actual KS systems. This way we can be certain that no extra structural syzygies appear⁶, and that all the observed gain is a result purely of the multi-homogeneous structure.

⁶ Such as those in [58].

Table 2. Comparison of quadratic, bilinear, and multigraded models for systems of equations with a shape coming from Kipnis-Shamir.

n	k	r	Quadratic		Bilinear		n'	Multi	
			Degree	columns	Degree	columns		Degree	columns
10	9	3	6	1947792	(5, 1)	44044	7	(1, 1, 1, 1, 1)	2560
10	9	4	8	95548245	(4, 2)	232375	9	(1, 1, 1, 1, 1, 1)	31250
10	9	5	10	2481256778	(10, 1)	2401828	10	(2, 1, 1, 1, 1, 1)	427680
10	9	6	13	101766230790	(15, 1)	32687600	9	(1, 3, 3, 3)	5927040
12	11	3	6	7059052	(4, 1)	38220	7	(1, 1, 1, 1, 1)	3072
12	11	4	7	99884400	(6, 1)	408408	9	(1, 1, 1, 1, 1, 1)	37500
12	11	5	9	6358402050	(9, 1)	6046560	9	(3, 1, 1, 1, 1)	471744
12	11	6	11	227692286640	(12, 1)	50026886	11	(3, 1, 1, 1, 1, 1)	6117748

The Reconciliation attack on SNOVA as multi-homogeneous system

Being a UOV type of system, SNOVA is susceptible to the reconciliation attack, although it is not competitive to the best known current attacks. As detailed in Example 1, we can form a multi-homogeneous system that can be solved using our Multi-homogeneous XL. Table 3 summarizes our estimates for the solving multi-degree of our algorithm for generic multi-homogeneous systems with parameters in a SNOVA regime. For example, the last line corresponds to Level I SNOVA parameters.

Table 3. Comparison of quadratic and multigraded models of the Reconciliation attack against SNOVA for various (v, m, o) . The size of the Macaulay matrix at the specified degree is given in $\log_2$ scale.

v	m	o	Quadratic		Multigraded	
			Degree	columns ($\log_2$)	Degree	columns ($\log_2$)
8	3	3	12	31	(3, 3, 6)	27
14	4	4	21	62	(4, 4, 4, 8)	54
15	4	4	27	75	(3, 6, 7, 10)	65
23	5	5	42	128	(7, 8, 8, 8, 9)	115
24	5	5	52	149	(3, 11, 11, 12, 13)	131

5 Implementation of the multi-homogeneous XL algorithm and optimization details

To verify the correctness of our algorithm and estimate bits of security of cryptographic schemes, we implemented the proposed Multi-homogeneous XL algorithm in C. While the algorithm described in 4 solves workable polynomial

systems by extracting right kernel vectors, our implementation takes a slightly different approach. In anticipation of future work which might require finding linear combinations of rows that eliminate selected columns, we implemented the block Lanczos algorithm [57] to extract left kernel vectors. The rest of this section provides details of our implementation and a computation cost analysis of the left kernel and the right kernel. Our result shows that in this case, searching in the left kernel incurs negligible extra computation overhead.

Algorithm 3 Multi-homogeneous XL for a degree set $\mathcal{D}$ (using the left kernel)

Input:

A set of polynomials $f_1, \dots, f_m \in \mathbb{F}_q[\mathbf{x}_1, \dots, \mathbf{x}_c]$

An order ideal $\mathcal{D}$ such that $\mathcal{P}(\mathbf{t})[\mathcal{D}] \leq 0$

Compute the multi-degree Macaulay matrix $\mathcal{M}_{\mathcal{D}}$

Split $\mathcal{M}_{\mathcal{D}}$ into 2 submatrices $\mathcal{M}_{\mathcal{D}}^{(l)}$ and $\mathcal{M}_{\mathcal{D}}^{(nl)}$

$\mathcal{M}_{\mathcal{D}}^{(nl)}$ contains columns for the constant and linear variables

$\mathcal{M}_{\mathcal{D}}^{(nl)}$ contains all the remaining columns not in $\mathcal{M}_{\mathcal{D}}^{(l)}$

$\text{Ker}(\mathcal{M})_{n+1} \leftarrow \mathbf{FindLeftKernelVectors}(\mathcal{M}_{\mathcal{D}}^{(nl)}, n+1)$ $\triangleright$ Find $n+1$ independent left kernel vectors

Compute the reduced matrix $\mathcal{M}_{\text{reduced}}$ from $\mathcal{M}_{\mathcal{D}}^{(l)}$ and $\text{Ker}(\mathcal{M})_{n+1}$

Reduce $\mathcal{M}_{\text{reduced}}$ with Gauss-Jordan elimination and return the solution

Algorithm 4 Find enough vectors in the left kernel of the given matrix

procedure FindLeftKernelVectors

Input:

Matrix $\mathcal{M}$

Number of kernel vectors to extract: n

Block size for the Lanczos algorithm: N

$S \leftarrow \emptyset$ $\triangleright$ Set of extracted left kernel vectors

while $|S| < n$ **do**

$S_{tmp} \leftarrow \mathbf{BlockLanczos}(\mathcal{M}, N)$

$S \leftarrow S \cup S_{tmp}$

Remove invalid kernel vectors from S

end while

return S

end procedure

Algorithm 5 The Block Lanczos Algorithm

procedure BlockLanczos
Input:

 Matrix $\mathcal{M} \in \mathbb{F}_q^{m \times n}$

 Block size: N

 Let A be the symmetric matrix $\mathcal{M}^T \cdot \mathcal{M}$

 Let I be the identity matrix $\in \mathbb{F}_q^{N \times N}$
 $V \leftarrow$ random matrix $\in \mathbb{F}_q^{m \times N}$
 $P \leftarrow$ zero matrix $\in \mathbb{F}_q^{m \times N}$
 $N_R \leftarrow 1$
while $N_R \geq 0$ **do**
 $\mathcal{M}^T V \leftarrow \mathcal{M}^T \cdot V$
 $\triangleright$ Sparse matrix multiplication

 $AV \leftarrow \mathcal{M} \cdot \mathcal{M}^T V$
 $\triangleright$ Sparse matrix multiplication

 $V^T AV \leftarrow (\mathcal{M}^T V)^T (\mathcal{M}^T V)$
 $\triangleright$ Gramian matrix

 $V^T A^2 V \leftarrow (AV)^T (AV)$
 $\triangleright$ Gramian matrix

 $N_R, D, W^{\text{inv}} \leftarrow \mathbf{FindInvertibleSubmatrix}(V^T AV)$
 $C \leftarrow W^{\text{inv}}(V^T A^2 V \cdot D + V^T AV(I - D))$
 $V_{\text{next}} \leftarrow AV \cdot D + V \cdot (I - D) - V \cdot C - P \cdot V^T AV \cdot D$
 $P_{\text{next}} \leftarrow V \cdot W^{\text{inv}} + P \cdot (I - D)$
 $V \leftarrow V_{\text{next}}$
 $P \leftarrow P_{\text{next}}$
end while
return V
 $\triangleright V$ contains N vectors in the left kernel of A
end procedure

5.1 Data structures and arithmetic

To compute the multi-degree Macaulay matrix, we multiply the base system of equations $f_1, \dots, f_m$ with all possible monomials allowed by the target multi-degree(s) (See Section 2.1). The Macaulay matrix is sparse therefore we simply store the coordinates and coefficients of non-zero entries. For efficient computation, the non-zero entries are stored in row-major order.

To compute a row uf_i in the Macaulay matrix, we multiply each monomial u_j in the polynomial f_i with the multiplying monomial u , which translates to mapping the monomial index of u_j to that of uu_j . One might be tempted to use a lookup table that stores the indices of all possible monomials for this purpose, but this approach might not be a good idea because the Macaulay matrix can have for example 2^{40} possible monomials. Each monomial index therefore requires at least 40 bits to store, and the minimal memory requirement for a naive implementation of such index lookup table is 5 terabytes. While one might be able to somewhat reduce the memory requirement, we choose to forgo the lookup table and compute monomial indices on the fly by taking advantage of the graded reverse lexicographical order. Our implementation can compute arbitrary multi-degree Macaulay matrix with column size (i.e. the number of all possible monomials) up to 2^{56} . The current upper bound exists due to the fact that we store both the column index and coefficient of a non-zero entry as a 64-bit integer. We use 8 bits for the coefficient and 56 bits for the index, hence the upper bound. If necessary, the upper bound can be further relaxed.

Similarly, we compute monomial indices on the fly for Macaulay matrices from gluing multi-degrees $\{\mathbf{D}_j\}$. We compute the set of all permitted multi-degrees $\{\mathbf{d}_i \mid \exists \mathbf{D}_j, \mathbf{d}_i \preceq \mathbf{D}_j\}$. Monomials of the same multi-degree $\mathbf{d}_i$ are grouped together and sorted by graded reverse lexicographical order. To derive the index of a monomial, we find its multi-degree $\mathbf{d}_i$ first, then compute the offset to the group of monomials of multi-degree $\mathbf{d}_i$, and finally the offset inside this group.

To eliminate arbitrary columns in the Macaulay matrix, we use the block Lanczos algorithm to look for vectors in the left kernel of the submatrix consisting of all the columns that we would like to eliminate. For this purpose, we need to traverse the Macaulay matrix along columns, but the non-zero entries of the Macaulay matrix are stored in the row-major order, which causes out-of-order memory access. We thus duplicate the submatrix to eliminate by storing its non-zero entries in the column-major order to ensure sequential memory access. Since the Lanczos algorithm does not modify the input submatrix, we only need to initialize the column-major storage once. The computation cost thereof is negligible and roughly as much extra memory is needed as the submatrix to eliminate. In other words, we trade memory for efficiency.

For dense matrices that appear in the block Lanczos algorithm, we design a special storage format. Since the basic unit of operation of block Lanczos is the computation of the product of a scalar and a vector, we store vectors in bitsliced format. For block size N , the first bits of the elements in a vector are grouped together into an N -bit integer, the second bits of the elements into another N -bit

integer, and so on. For example, let a vector $\mathbf{c} = \{c_0, c_1, \dots, c_{N-1}\} \in \mathbb{F}_{16}^N$, where $c_i = (c_{i,3}c_{i,2}c_{i,1}c_{i,0})_2$, we store $\mathbf{c}$ as in Figure 1:

N -bit integer	
1 st bit	$(c_{N-1,0}c_{N-2,0} \cdots c_{1,0}c_{0,0})_2$
2 nd bit	$(c_{N-1,1}c_{N-2,1} \cdots c_{1,1}c_{0,1})_2$
3 rd bit	$(c_{N-1,2}c_{N-2,2} \cdots c_{1,2}c_{0,2})_2$
4 th bit	$(c_{N-1,3}c_{N-2,3} \cdots c_{1,3}c_{0,3})_2$

Fig. 1. Bitslicing a vector of N elements over $\mathbb{F}_{16}$

We choose N according to the size of registers provided by the hardware. For example, 64 for plain x86_64 architectures, and greater for architectures which support larger registers. Our implementation supports $N = 64, 128, 256, 512$. In our experiments, $N = 64$ or 128 leads to the best performance.

One benefit of this bitslicing format is that addition of vectors of N elements in $\mathbb{F}_{2^n}$ becomes n XOR operations. Take $\mathbb{F}_{16}$ as an example, we can view each scalar $c = (c_3c_2c_1c_0)_2 \in \mathbb{F}_{16}$ as a polynomial $c_3x^3 + c_2x^2 + c_1x^1 + c_0$. Addition of two scalars thus can be interpreted as addition of two such polynomials, with coefficients over $\mathbb{F}_2$. Consequently we can completely avoid field reduction. Similarly, by interpreting a scalar as a polynomial, multiplication of a vector of N elements with the scalar can be implemented with only AND and XOR instructions, and thereby avoiding branch misprediction. The number of machine instructions required for a field multiplication is also greatly reduced. For example in our implementation, roughly 15 machine instructions are needed to multiply a scalar with a vector of 64 elements in $\mathbb{F}_{16}$, and some of these instructions will even be executed in parallel at the microarchitectural level.

5.2 Implementation of the Block Lanczos algorithm

In procedure **FindInvertibleSubmatrix** of Algorithm 5, we first apply Gaussian elimination on the input matrix M to find its rank and independent columns. From the independent columns, we construct the diagonal matrix D as:

$$\begin{cases} D_{i,i} = 1 & \text{if the } i^{\text{th}} \text{ column of } M \text{ is independent} \\ D_{i,i} = 0 & \text{if the } i^{\text{th}} \text{ column of } M \text{ is not independent} \\ D_{i,j} = 0 & \text{for } i \neq j \end{cases}$$

Subsequently we compute the inverse of the invertible submatrix $M \cdot D$ as M^{inv} . Note that M^{inv} is zero outside the rows and columns selected by D . The rank, D , and M^{inv} are then returned to the caller.

While we have already analyzed the asymptotic complexity of our proposed algorithm in Section 4, we can derive better estimation of bits of security by taking the machine instructions into consideration. We identified that the basic computation that is repeated over and over again is the multiplication of a

constant $c \in \mathbb{F}_q$ with a vector $\in \mathbb{F}_q^N$. Consequently, the cost of extracting kernel vectors with our algorithm can be estimated by defining such multiplication as the basic unit of operation and its cost as B_{mul} , which is N field multiplications. As an example, B_{mul} is roughly 15 machine instructions for every 64 field elements in $\mathbb{F}_{16}$ (see Section 5.1) in our implementation.

Recall that the input matrix for the block Lanczos algorithm $M \in \mathbb{F}_q^{m \times n}$ is a submatrix of the full Macaulay matrix $\mathcal{M}_{\mathcal{D}} \in \mathbb{F}_q^{m_f \times n_f}$, and the block size is N . Let the row density of M and $\mathcal{M}_{\mathcal{D}}$ be ρ and ρ_f , i.e. the number of non-zero entries per row (the numbers of non-zero entries of M and $\mathcal{M}_{\mathcal{D}}$ are thus ρm and $\rho_f m_f$ respectively), rank of M be R , rank of $\mathcal{M}_{\mathcal{D}}$ be R_f , the block Lanczos algorithm expects to extract N left kernel vectors of MM^T in $\approx \frac{R}{N}$ iterations (see [49] for the exact formula), and each iteration involves computation listed in Table 4.

Operation	Left kernel of MM^T	Right kernel of $\mathcal{M}_{\mathcal{D}}^T \mathcal{M}_{\mathcal{D}}$
Sparse matrix multiplication	$2\rho m B_{\text{mul}}$	$2\rho_f m_f B_{\text{mul}}$
Gramian matrix computation	$(m+n)N B_{\text{mul}}$	$(m_f+n_f)N B_{\text{mul}}$
Matrix multiplication	$(N^2+3mN)B_{\text{mul}}$	$(N^2+3n_fN)B_{\text{mul}}$
Gaussian Elimination	$N^2 B_{\text{mul}}$	the same

Table 4. Computation cost for each block Lanczos iteration

The cost of extracting N left kernel vectors is thus expected to be $\frac{R}{N}(2\rho m + 4mN + nN + 2N^2)B_{\text{mul}}$. The number of kernel vectors to extract depends on the rank of the matrix which we would like to extract from the Macaulay matrix. For our use case, it is $k+r \cdot c+1$. Therefore we expect to call the block Lanczos procedure $\lceil \frac{k+r \cdot c+1}{N} \rceil$ times, which is just 1 for the instances we tested. While the extracted kernel vectors are in the left kernel of A and might not be in the left kernel of M , heuristically the intersection between the left kernel of M and A will be sufficiently large to avoid such cases, provided that the left kernel of M is large enough. A justification can be found in [57]. In our experiments, we have not encountered a left kernel vector of A that is not in the left kernel of M .

Note that for a Macaulay matrix derived from a Kipnis-Shamir matrix of a MinRank instance, we can estimate its row density as $k \cdot r + k + r + 1$ since each row has at most this number of non-zero entries.

Alternatively, we can solve the system directly by extracting right kernel vectors of $\mathcal{M}_{\mathcal{D}}^T \mathcal{M}_{\mathcal{D}}$ and the cost would be $\frac{R_f}{N}(2\rho_f m_f + m_f N + 4n_f N + 2N^2)B_{\text{mul}}$. Comparing to the left kernel, the main difference lies in the cost of matrix multiplication since in our use case $\rho_f \approx \rho$, $n_f \approx n$, and $m_f = m$, which would be higher for left kernel if $m > n_f$. To reduce the computation cost, we can keep just enough rows by randomly sampling roughly R rows when extracting left kernel vectors. Another difference is that one right kernel vector is sufficient while $k+r \cdot c+1$ left kernel vectors are needed.

For small fields, such as $\mathbb{F}_2, \mathbb{F}_3, \mathbb{F}_5$, one can further reduce the computation cost with, e.g. the Method of 4 Russians [5]. We did not implement this op-

timization because our field is $\mathbb{F}_{16}$, which does not seem to benefit from this technique.

5.3 Alternative algorithm for finding kernel vectors

Since the Macaulay matrix is extremely sparse by nature, one should take advantage of sparse multiplication. In our implementation, we use the block Lanczos algorithm to extract kernel vectors. One could alternatively use the block Wiedemann algorithm[60]. One prominent difference is that it requires three multiplications with a large sparse matrix per iteration, unlikely block Lanczos which just requires two. The block Wiedemann algorithm however allows better parallelization and is more suitable for distributed computation. We chose the block Lanczos algorithm because we do not have access to server clusters.

6 Experimental results

We present several applications of our algorithm in various cryptanalytic attacks. We also experimentally demonstrate the performance of our algorithm. To cover actual cryptographic parameters, we hybridize the algorithm. I.e. we fix unknowns to reduce parameters such that our algorithm becomes applicable to the reduced parameter set. We then estimate the total computation cost by simulation. The experiments were conducted on an office laptop with a quad-core Intel CPU from 2017 (i7-8650U) and 32GB of memory. The implementation is optimized with both AVX2 and AVX512 instruction sets. If the CPU supports the newer AVX512 instructions, the execution time can be further reduced roughly by half. We measured execution time by generating random workable polynomial systems with the target parameter set. The variation of the measured execution time is negligible. If not stated, the unit of the execution time listed in the following tables is second. For lengthy experiments, we extrapolate their execution time from smaller MinRank instances and we mark them with an asterisk in the tables. We also list the amount of memory required to store the matrices.

We compare our algorithm with MinRank instances listed in Table 1 and Table 2 in the work of Verbel et al[58]. They denote the number of kernel vectors to use as κ , which we keep in Table 5 and Table 6.

All parameters are the same except the field and the number of matrices in the MinRank instance. Since Verbel et al consider homogeneous systems while we consider inhomogeneous systems, the computation cost of solving a system with parameters $(m, n, k + 1, r)$ in their paper is equivalent to the cost of solving a system with parameter (m, n, k, r) in our work. Besides this, they use $\mathbb{F}_{13}$ and we use $\mathbb{F}_{16}$. The field size affects the cost of fixing unknowns and the basic computation unit B_{mul} (See Section 5.2). Since the field sizes do not differ much, we expect that the estimated bits of security from our experiments to be slightly higher than that of the MinRank instances with the same parameters over $\mathbb{F}_{13}$.

⁷ Assuming $D_{\text{siv}} = 6$

Macaulay matrix constructed by multiplying only kernel variables[58]					
r	2	3	4	5	6
D_{solv}	3	4	4	5	6
Matrix size (F4)	2217	24582	38586	341495	> 2035458
Matrix size (XL)	1530	20240	29250	237510	1187550 ⁷
Multi-homogeneous XL (our algorithm)					
(m, n', k) after fixing	(10, 5, 10)	(10, 6, 9)	(10, 9, 9)	(10, 10, 9)	(10, 9, 9)
Multi-degree	(1, 1, 1, 2)	(1, 1, 1, 1, 1)	(1, 1, 1, 1, 1, 1)	(2, 1, 1, 1, 1, 1)	(1, 3, 3, 3)
Matrix size	577	2560	31250	427680	5927040
Memory (MB)	0.41	1.92	28.17	652.24	7051.2
Bits of security	2^{21}	2^{25}	2^{32}	2^{40}	2^{47}
Execution time	0.02 sec	0.09 sec	8.98 sec	2446.75 sec	≈ 3 days*

Table 5. Results of solving MinRank instances ($m = 10, n = 10, k = 10, r = 2 \sim 6$)

Ideally one would compare the execution time but due to the fact that there is no specification of the hardware they used for their experiments, we cannot do so. Their complexity analysis also seems quit coarse and the estimated complexity from their formula is much higher than our estimation. Consequently, we simply compare the size of the Macaulay matrices (i.e. the number of columns) generated with their approach and ours. We highlight the fact that Verbel et al[58] took advantage of degree falls and constructed Macaulay matrix based on the solving degree. In comparison, our approach is *generic*. Note that Verbel et al did not solve ($m = 10, n = 10, k = 10, r = 4 \sim 6$) with their approach.

Macaulay matrix constructed by multiplying only kernel variables[58]				
r	3	4	5	6
D_{solv}	4	4	4	5
κ	4	5	4	5
Matrix size (XL)	5460	21252	21252	556512
Memory (MB)	N/A	104	352	20928
Multi-homogeneous XL (our algorithm)				
(m, n', k) after fixing	(12, 7, 11)	(12, 9, 11)	(12, 9, 11)	(12, 11, 11)
Multi-degree	(1, 1, 1, 1, 1)	(1, 1, 1, 1, 1, 1)	(3, 1, 1, 1, 1)	(3, 1, 1, 1, 1, 1)
Matrix size	3072	37500	471744	6117748
Memory (MB)	2.67	38.84	952.22	15096.75
Bits of security	2^{26}	2^{33}	2^{41}	2^{49}
Execution time	0.33 sec	12.99 sec	3613.67 sec	≈ 12 days*

Table 6. Results of solving MinRank instances ($m = 12, n = 12, k = 12, r = 3 \sim 6$)

Because of the additional structure, the size of our matrix $\mathcal{M}_{\mathcal{D}}$ is significantly smaller than a matrix of degree $D_{\text{solv}} \leq$ the total degree of $\mathcal{D}$. For example, in Table 5, the size of our matrices with multi-degree (1, 1, 1, 2) and (1, 1, 1, 1, 1) is

much smaller than matrices of degree $D_{\text{siv}} \leq 5$ generated with their approach. With our algorithm, all these MinRank instances can be solved in a reasonable amount of time with a 10-year-old office laptop.

Multi-homogeneous XL against MIRATH We also analyzed the complexity of attacking the Mirath signature scheme [1] with our algorithm and the result is listed in Table 7.

Mirath parameter sets	Mirath-1a	Mirath-3a	Mirath-5a
Original Parameters (m, n, r, k)	(16, 16, 4, 143)	(19, 19, 5, 195)	(22, 22, 6, 255)
After fixing vectors (m, n, r, k)	(16, 8, 4, 15)	(19, 10, 5, 24)	(22, 11, 6, 13)
Multi-degree	(1, 2, 2, 2)	(3, 1, 1, 1, 1, 1)	(1, 2, 2, 1)
Matrix size	54000	22744800	76832
Memory (MB)	71	88901	122
Bits of security claimed[1]	2^{158}	2^{225}	2^{301}
Bits of security (XL)	2^{176}	2^{245}	2^{309}
Bits of security (multi-homogeneous XL)	2^{162}	2^{232}	2^{299}
Bits of security due to fixing vectors	2^{128}	2^{180}	2^{264}
Remaining security after fixing (multi-homogeneous XL)	2^{34}	2^{52}	2^{35}
Execution time (single iteration)	29.27 sec	≈ 96 days*	60.89 sec

Table 7. Complexity analysis of attacking Mirath[1]

Our estimation of bits of security is quite accurate. We use B_{mul} as the unit, which is roughly 15 machine instructions for the multiplication of a scalar with a vector of 64 elements in $\mathbb{F}_{16}$. The CPU used for the experiments (i7-8650U) operates at 3.4GHz when all 4 cores are utilized. Therefore in the span of 29.27 seconds, the total number of instructions executed is roughly: $29.27 \cdot 4 \cdot 3.4 \cdot 10^9 = 2^{38.53}$, which is close to the estimated bits of security: $2^{34} B_{\text{mul}} = 2^{34} \cdot 15 \cdot \frac{128}{64} = 2^{38.9}$. Here B_{mul} is 30 instructions because the block size is 128 instead of 64. In comparison, the bits of security listed in the specification of Mirath[1] is somewhat lower, which might suggest that the security analysis was done conservatively. We can also notice that the reduction of bits of security from using the plain XL to using our approach is more significant for higher degrees. In other words, Multi-homogeneous XL would be more effective for Macaulay matrices with a larger total degree.

References

1. Adj, G., Aragon, N., Barbero, S., Bardet, M., Bellini, E., Bidoux, L., Chi-Dominguez, J.J., Dyseryn, V., Esser, A., Feneuil, T., et al.: Mirath signature scheme (2025), https://pqc-mirath.org/assets/downloads/mirath_v2.1.0.pdf

2. Arora, S., Ge, R.: New algorithms for learning in presence of errors. In: ICALP (1). pp. 403–415 (2011)
3. B., B.: Ein algorithmus zum auffinden der basiselemente des restklassenringes nach einem nulldimensionalen polynomideal. Ph. D. Thesis, Math. Inst., University of Innsbruck (1965), <https://cir.nii.ac.jp/crid/1570291224270729344>
4. Bard, G.V.: On the Rapid Solution of Systems of Polynomial Equations over Low-Degree Extension Fields of $\text{GF}(2)$, via SAT-Solvers. 8th Central European Conf. on Cryptography (2008)
5. Bard, G.V.: The method of four russians. In: Algebraic Cryptanalysis, pp. 133–158. Springer (2009)
6. Bard, G.V., Courtois, N.T., Jefferson, C.: Efficient Methods for Conversion and Solution of Sparse Systems of Low-Degree Multivariate Polynomials over $\text{GF}(2)$ via SAT-Solvers. Cryptology ePrint Archive, Report 2007/024 (2007), <http://eprint.iacr.org/>
7. Bardet, M., Briaud, P., Bros, M., Gaborit, P., Neiger, V., Ruatta, O., Tillich, J.: An algebraic attack on rank metric code-based cryptosystems. In: Canteaut, A., Ishai, Y. (eds.) Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10-14, 2020, Proceedings, Part III. Lecture Notes in Computer Science, vol. 12107, pp. 64–93. Springer (2020). https://doi.org/10.1007/978-3-030-45727-3_3, https://doi.org/10.1007/978-3-030-45727-3_3
8. Bardet, M., Bros, M., Cabarcas, D., Gaborit, P., Perlner, R., Smith-Tone, D., Tillich, J.P., Verbel, J.: Improvements of algebraic attacks for solving the rank decoding and minrank problems. In: Moriai, S., Wang, H. (eds.) Advances in Cryptology – ASIACRYPT 2020. pp. 507–536. Springer International Publishing, Cham (2020)
9. Bardet, M., Faugère, J.C., Salvy, B.: On the complexity of Gröbner basis computation of semi-regular overdetermined algebraic equations. In: Proc. International Conference on Polynomial System Solving (ICPSS). pp. 71–75 (2004)
10. Bardet, M., Faugère, J., Salvy, B., Spaenlehauer, P.: On the complexity of solving quadratic Boolean systems. Journal of Complexity **29**(1), 53–75 (2013), www-polsys.lip6.fr/~jcf/Papers/BFSS12.pdf
11. Benadjila, R., Feneuil, T., Rivain, M.: MQ on my Mind: Post-Quantum Signatures from the Non-Structured Multivariate Quadratic Problem . In: 2024 IEEE 9th European Symposium on Security and Privacy (EuroS&P). pp. 468–485. IEEE Computer Society, Los Alamitos, CA, USA (Jul 2024). <https://doi.org/10.1109/EuroSP60621.2024.00032>, <https://doi.ieeecomputersociety.org/10.1109/EuroSP60621.2024.00032>
12. Berbain, C., Gilbert, H., Patarin, J.: QUAD: A Practical Stream Cipher with Provable Security. In: Vaudenay, S. (ed.) EUROCRYPT '06. Lecture Notes in Computer Science, vol. 4004, pp. 109–128. Springer (2006)
13. Bettale, L., Faugère, J.C., Perret, L.: Hybrid approach for solving multivariate systems over finite fields. Journal of Mathematical Cryptology **volume 3**(issue 3), 177–197 (2009), <http://www-polsys.lip6.fr/~jcf/Papers/JMC2.pdf>
14. Beullens, W.: Sigma Protocols for MQ, PKP and SIS, and Fishy Signature Schemes. In: EUROCRYPT (3). Lecture Notes in Computer Science, vol. 12107, pp. 183–211. Springer (2020)
15. Beullens, W.: Improved cryptanalysis of uov and rainbow. In: Canteaut, A., Standaert, F.X. (eds.) Advances in Cryptology – EUROCRYPT 2021. pp. 348–373. Springer International Publishing, Cham (2021)

16. Beullens, W.: Mayo: Practical post-quantum signatures from oil-and-vinegar maps. In: AlTawy, R., Hülsing, A. (eds.) *Selected Areas in Cryptography*. pp. 355–376. Springer International Publishing, Cham (2022)
17. Beullens, W., Chen, M.S., Hung, S.H., Kannwischer, M.J., Peng, B.Y., Shih, C.J., Yang, B.Y.: Oil and Vinegar: Modern Parameters and Implementations. *Cryptology ePrint Archive, Report 2023/059* (Jun 2023), <https://ia.cr/2023/059>, accepted for publication at TCHES’23
18. Björklund, A., Kaski, P., Williams, R.: Solving Systems of Polynomial Equations over $\text{GF}(2)$ by a Parity-Counting Self-Reduction. In: Baier, C., Chatzigiannakis, I., Flocchini, P., Leonardi, S. (eds.) *46th International Colloquium on Automata, Languages, and Programming (ICALP 2019)*. Leibniz International Proceedings in Informatics (LIPIcs), vol. 132, pp. 26:1–26:13. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany (2019). <https://doi.org/10.4230/LIPIcs.ICALP.2019.26>, <http://drops.dagstuhl.de/opus/volltexte/2019/10602>
19. Buchberger, B.: Ein Algorithmus zum Auffinden der Basiselemente des Restklassenringes nach einem nulldimensionalen Polynomideal. Ph.D. thesis, University of Innsbruck (1965)
20. Cabarcas, D., Li, P., Verbel, J., Villanueva-Polanco, R.: Improved attacks for snova by exploiting stability under a group action. In: Tauman Kalai, Y., Kamara, S.F. (eds.) *Advances in Cryptology – CRYPTO 2025*. pp. 358–389. Springer Nature Switzerland, Cham (2025)
21. Chen, M.S., Hülsing, A., Rijneveld, J., Samardjiska, S., Schwabe, P.: From 5-pass $\mathcal{MQ}$ -based identification to $\mathcal{MQ}$ -based signatures. In: Cheon, J.H., Takagi, T. (eds.) *Advances in Cryptology – ASIACRYPT 2016*. vol. 10032, pp. 135–165 (2016), <http://eprint.iacr.org/2016/708>
22. Chen, M., Hülsing, A., Rijneveld, J., Samardjiska, S., Schwabe, P.: SOFIA: $\mathcal{MQ}$ -based signatures in the QROM. In: Abdalla, M., Dahab, R. (eds.) *Public-Key Cryptography - PKC 2018 - 21st IACR International Conference on Practice and Theory of Public-Key Cryptography*, Rio de Janeiro, Brazil, March 25–29, 2018, Proceedings, Part II. *Lecture Notes in Computer Science*, vol. 10770, pp. 3–33. Springer (2018). https://doi.org/10.1007/978-3-319-76581-5_1, https://doi.org/10.1007/978-3-319-76581-5_1
23. Coppersmith, D.: Solving homogeneous linear equations over $\text{gf}(2)$ via block wiede-mann algorithm. *Mathematics of Computation* **62**(205), 333–350 (1994)
24. Courtois, N., Klimov, A., Patarin, J., Shamir, A.: Efficient algorithms for solving overdefined systems of multivariate polynomial equations. In: *International Conference on the Theory and Applications of Cryptographic Techniques*. pp. 392–407. Springer (2000)
25. Ding, J., Schmidt, D.: Rainbow, a New Multivariable Polynomial Signature Scheme. In: Ioannidis, J., Keromytis, A.D., Yung, M. (eds.) *ACNS. Lecture Notes in Computer Science*, vol. 3531, pp. 164–175 (2005)
26. Ding, J., Yang, B.Y., Chen, C.H.O., Chen, M.S., Cheng, C.M.: New Differential-Algebraic Attacks and Reparametrization of Rainbow. In: Bellovin, S.M., Gennaro, R., Keromytis, A.D., Yung, M. (eds.) *ACNS. Lecture Notes in Computer Science*, vol. 5037, pp. 242–257 (2008)
27. Dinur, I.: Improved Algorithms for Solving Polynomial Systems over $\text{GF}(2)$ by Multiple Parity-Counting. In: *Proceedings of the Thirty-Second Annual ACM-SIAM Symposium on Discrete Algorithms*. p. 2550–2564. SODA ’21, Society for Industrial and Applied Mathematics, USA (2021)

28. Faugère, J.C., Gianni, P., Lazard, D., Mora, T.: Efficient computation of zero-dimensional Gröbner bases by change of ordering. *J. Symb. Comput.* **16**, 329–344 (October 1993). <https://doi.org/10.1006/jsco.1993.1051>
29. Faugère, J.C.: A new efficient algorithm for computing Gröbner bases (F4). *Journal of Pure and Applied Algebra* **139**, 61–88 (1999), <http://www.polysys.lip6.fr/~jcf/Papers/F99a.pdf>
30. Faugère, J.C.: A new efficient algorithm for computing Gröbner bases without reduction to zero (F5). In: *Proceedings of the 2002 International Symposium on Symbolic and Algebraic Computation ISSAC*. pp. 75–83. ACM Press (2002)
31. Faugère, J., Gauthier-Umaña, V., Otmani, A., Perret, L., Tillich, J.: A Distinguisher for High-Rate McEliece Cryptosystems. *IEEE Trans. Inf. Theory* **59**(10), 6830–6844 (2013). <https://doi.org/10.1109/TIT.2013.2272036>, <https://doi.org/10.1109/TIT.2013.2272036>
32. Faugère, J.C., Joux, A.: Algebraic cryptanalysis of Hidden Field Equation (HFE) cryptosystems using Gröbner bases. In: Boneh, D. (ed.) *Advances in Cryptology - CRYPTO 2003*. LNCS, vol. 2729, pp. 44–60. Springer (2003)
33. Faugère, J.C., dit Vehel, F.L., Perret, L.: Cryptanalysis of MinRank. In: Wagner, D. (ed.) *CRYPTO. Lecture Notes in Computer Science*, vol. 5157, pp. 280–296. Springer (2008)
34. Faugère, J.C., Safey El Din, M., Spaenlehauer, P.J.: Gröbner bases of bihomogeneous ideals generated by polynomials of bidegree (1,1): Algorithms and complexity. *Journal of Symbolic Computation* **46**(4), 406–437 (2011). <https://doi.org/https://doi.org/10.1016/j.jsc.2010.10.014>, <https://www.sciencedirect.com/science/article/pii/S0747717110001902>
35. Faugère, J.C., Safey El Din, M., Spaenlehauer, P.J.: On the complexity of the generalized minrank problem. *Journal of Symbolic Computation* **55**, 30–58 (2013). <https://doi.org/https://doi.org/10.1016/j.jsc.2013.03.004>, <https://www.sciencedirect.com/science/article/pii/S0747717113000485>
36. Furue, H., Ikematsu, Y., Kiyomura, Y., Takagi, T.: A new variant of unbalanced oil and vinegar using quotient ring: Qr-uov. In: Tibouchi, M., Wang, H. (eds.) *Advances in Cryptology – ASIACRYPT 2021*. pp. 187–217. Springer International Publishing, Cham (2021)
37. Gaborit, P., Ruatta, O., Schrek, J.: On the complexity of the rank syndrome decoding problem. *IEEE Trans. Inf. Theory* **62**(2), 1006–1019 (2016). <https://doi.org/10.1109/TIT.2015.2511786>, <https://doi.org/10.1109/TIT.2015.2511786>
38. Garey, M.R., Johnson, D.S.: *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman (1979)
39. Joux, A., Vitse, V.: A Crossbred Algorithm for Solving Boolean Polynomial Systems. In: Kaczorowski, J., Pieprzyk, J., Pomykała, J. (eds.) *Number-Theoretic Methods in Cryptology*. pp. 3–21. Springer International Publishing, Cham (2018)
40. Kipnis, A., Patarin, J., Goubin, L.: Unbalanced Oil and Vinegar Signature Schemes. In: *Advances in Cryptology – EUROCRYPT ’99*. *Lecture Notes in Computer Science*, vol. 1592, pp. 206–222. Springer (1999)
41. Kipnis, A., Shamir, A.: Cryptanalysis of the HFE Public Key Cryptosystem by Re-linearization. In: *CRYPTO ’99*. LNCS, vol. 1666, pp. 19–30. Springer (1999)
42. Koichi Sakumoto, Shirai, T., Hiwatari, H.: Public-key identification schemes based on multivariate quadratic polynomials. In: Rogaway, P. (ed.) *Advances in Cryptology – CRYPTO 2011*. vol. 6841, pp. 706–723 (2011), <https://www.iacr.org/archive/crypto2011/68410703/68410703.pdf>

43. Kreuzer, M., Robbiano, L.: Basic tools for computing in multigraded rings. In: Herzog, J., Vuletescu, V. (eds.) *Commutative Algebra, Singularities and Computer Algebra*. pp. 197–216. Springer Netherlands, Dordrecht (2003)
44. Kreuzer, M., Robbiano, L.: *Computational Commutative Algebra 2* (01 2005). <https://doi.org/10.1007/3-540-28296-3>
45. Lazard, D.: Gröbner bases, gaussian elimination and resolution of systems of algebraic equations. In: *European Conference on Computer Algebra*. pp. 146–156. Springer (1983)
46. Lokshtanov, D., Paturi, R., Tamaki, S., Williams, R.R., Yu, H.: Beating brute force for systems of polynomial equations over finite fields. In: Klein, P.N. (ed.) *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2017, Barcelona, Spain, Hotel Porta Fira, January 16-19*. pp. 2190–2202. SIAM (2017). <https://doi.org/10.1137/1.9781611974782.143>, <https://doi.org/10.1137/1.9781611974782.143>
47. Massacci, F., Marraro, L.: Logical Cryptanalysis as a SAT Problem. *J. Autom. Reasoning* **24**(1/2), 165–203 (2000)
48. Mohamed, M.S.E., Mohamed, W.S.A.E., Ding, J., Buchmann, J.: MXL2: Solving Polynomial Equations over GF(2) Using an Improved Mutant Strategy. In: Buchmann, J., Ding, J. (eds.) *PQCrypto. Lecture Notes in Computer Science*, vol. 5299, pp. 203–215. Springer (2008)
49. Montgomery, P.L.: A block lanczos algorithm for finding dependencies over gf(2). In: Guillou, L.C., Quisquater, J.J. (eds.) *Advances in Cryptology — EUROCRYPT ’95*. pp. 106–120. Springer Berlin Heidelberg, Berlin, Heidelberg (1995)
50. NAKAMURA, S., TANI, Y., FURUE, H.: Lifting approach against the snova scheme. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences* **E108.A**(10), 1373–1381 (2025). <https://doi.org/10.1587/transfun.2024EAP1124>
51. Nakamura, S., Wang, Y., Ikematsu, Y.: Analysis on the MinRank attack using kipnis-shamir method against rainbow. *Cryptology ePrint Archive*, Paper 2020/908 (2020), <https://eprint.iacr.org/2020/908>
52. NIST (National Institute for Standards and Technology): *Post-Quantum Cryptography Standardization* (2017), uRL: <https://csrc.nist.gov/Projects/Post-Quantum-Cryptography>
53. Perlner, R., Smith-Tone, D.: Rainbow Band Separation is Better than we Thought. *Cryptology ePrint Archive*, Report 2020/702 (2020), <https://eprint.iacr.org/2020/702>
54. Ran, L., Samardjiska, S.: Rare structures in tensor graphs. In: Chung, K.M., Sasaki, Y. (eds.) *Advances in Cryptology – ASIACRYPT 2024*. pp. 66–96. Springer Nature Singapore, Singapore (2025)
55. Thomae, E.: *About the Security of Multivariate Quadratic Public Key Schemes*. Ph.D. thesis, Ruhr-University Bochum, Germany (2013), https://www.iacr.org/phds/116_EnricoThomae_AboutSecurityMultivariateQuadr.pdf
56. Thomae, E., Wolf, C.: Solving underdetermined systems of multivariate quadratic equations revisited. In: *International workshop on public key cryptography*. pp. 156–171. Springer (2012)
57. Thomé, E.: A modified block lanczos algorithm with fewer vectors. *arXiv preprint arXiv:1604.02277* (2016)
58. Verbel, J., Baena, J., Cabarcas, D., Perlner, R., Smith-Tone, D.: On the complexity of “superdetermined” minrank instances. In: Ding, J., Steinwandt, R. (eds.) *Post-Quantum Cryptography*. pp. 167–186. Springer International Publishing, Cham (2019)

59. Wang, L.C., Tseng, P.E., Kuan, Y.L., Chou, C.Y.: A simple noncommutative UOV scheme. Cryptology ePrint Archive, Paper 2022/1742 (2022), <https://eprint.iacr.org/2022/1742>
60. Wiedemann, D.: Solving sparse linear equations over finite fields. IEEE transactions on information theory **32**(1), 54–62 (1986)
61. Yang, B.Y., Chen, J.M.: All in the XL Family: Theory and Practice. In: Park, C., Chee, S. (eds.) ICISC '04. Lecture Notes in Computer Science, vol. 3506, pp. 67–86. Springer (2004)
62. Yang, B., Chen, J.: Theoretical analysis of XL over small fields. In: Information Security and Privacy. pp. 277–288 (2004). https://doi.org/10.1007/978-3-540-27800-9_24, http://dx.doi.org/10.1007/978-3-540-27800-9_24, <http://www.iis.sinica.edu.tw/papers/byyang/2386-F.pdf>